Software Project Management

Introduction

What is a project?

What's a project?

- PMI (Project Management Institute) definition
  - *A project is a temporary endeavor undertaken to create a unique product or service*
- Description: A sequence of connected and related activities ***(requirement engineering, system engineering, coding, testing, documentation, controlling, …)*** that must be completed by a specific time, within budget, and according to specification.
- Progressively elaborated
  - With repetitive elements
- A project manager
  - Analogy: conductor, coach, captain

Project vs. Program Management

What's a 'program'?

- Mostly differences of scale
- Often a number of related projects
- Longer than projects
- Definitions vary
- Example: Program Manager for MS Office
  - Includes projects: Spelling, drawing, Word, Excel, PowerPoint, etc.

Project characteristics

- A project is unique
  - A Project creates unique deliverables: product, service, or result
  - Even technically-identical projects are distinctly unique, due to internal or external contingent factors
- Projects are temporary
  - Every project has a definite beginning and a definite end
  - Project end may be reached through success, qualified success, failure, or redundancy
  - Projects need not be of short duration, but they are of finite duration

Project characteristics

- Projects have customer-specified performance criteria
  - The customer defines these criteria in the form of requirements
  - We will see that **requirements** are one of the **most critical characteristics** of a project that contributes to its **success or failure**
- Projects consume resources
  - Personnel resources
  - Physical resources
  - Monetary resources

Projects *vs. operations/production*

Similarities

Both Projects & Operations

- Consume resources
- Are constrained by budget, time, resources, etc.
- Are planned, executed, & controlled

Projects

- Are unique
- Are temporary
- *Example*: Developing v1.0 of a new cloud-based software product (*e.g.*, Adobe Creative Cloud)

Differences

Operations

- Are repetitive
- Are on-going
- *Example*: Maintenance and delivery of cloud-based software product

Characteristics of IT (Information Technology)

- IT encompasses all forms of technology used to create, store, exchange, and use information in various forms
- ‘Information’ includes conventional data, voice, images, multimedia, etc.
- Of central importance are computer, software, and communications technologies
- Virtually all significant projects are now distributed: networks of computers communicate only via message passing
- Distribution poses additional challenges in IT projects: reliability, availability, security, and information synchronization

How do IT projects differ from ‘ordinary’ projects?

- IT products and services possess greater complexity
  - IT products and services are basically complex by nature
  - Computing + communication + diverse data ⇒ complexity
- IT projects have tight schedules
  - Tight schedules are the norm in IT development
  - Scheduling is aggravated by a pervasive ‘rush to market’ mentality
- IT is an integral part of enterprise infrastructure
  - IT is no longer an auxiliary element of the enterprise
  - A business’ success is often critically dependent upon IT support
- IT is permeated by quickly-changing technology
  - Technology evolves (and may even become obsolete) during the lifetime of a major project

Project characteristics

- Four characteristics of projects:
  - finite time
  - people assigned
  - clear roles and responsibilities
  - things to deliver
- Have you ever had this feeling about a project?
  - not enough time
  - too few people
  - people not sure what they should be doing
  - too much to do

How Are Software Projects Different?

- Consists of hardware and software.
- Software is [usually] custom written and one-of-a-kind.
- Hard to determine progress. [One can see how far the Freedom Tower (World Trade Center) has progressed. One cannot see how far a piece of software has progressed]
- Difficult to estimate schedule.
- Difficult to determine cost.

What is Project Management?

- **Project management** is “the application of knowledge, skills, tools and techniques to project activities to meet project requirements”
- Software Project Management is the art to define, plan, execute, and monitor the activities that will bring **software products** to existence.
- Project managers strive to meet the **triple constraint** by balancing project scope, time, and cost goals

Project management processes

- Regardless of the type of project lifecycle, project management encompasses the following process groups, shown with some representative tasks:
  1. **Initiating/Define** – Scope the project; Charter the project; identify stakeholders
  2. **Planning** – Develop the project plan**.** Collect requirements; identify schedule; plan scope, cost, quality, human resource, risk, and procurement management
  3. **Executing** – Launch the plan**.** Direct and manage project work; perform quality assurance; manage and develop project team; conduct procurements
  4. **Monitoring and Controlling** – Monitor project progress. Monitor and control project work; manage scope change; monitor and control schedule; control quality; control risks; control procurements
  5. **Closing** – Close out the project: Close project; close procurements

Software Project

Quality Engineering Principle:

- The quality of the software system is controlled by the quality of the process used to produce that software.

Quality Management Principle:

- Document the process
- Measure the process
- Improve the process based on the measurement

Software Development Process

- What is the software development process?
  - A process is a **set of documented** procedures, methods, practices, and tools used to produce a software product.
- The process will answer the following:
  - What to do? **Tasks/activities**
  - How to do it? **Procedure/practice**
  - When to do it? **Sequence of activities**
  - What are the artifacts? **(input/output)**

Software Project

- If the programmer and designer follow the **process,** then the artifacts they produce will be
  - Predictable
  - Based on the requirements
  - Easy to maintain and control
  - Consistent with the writing style
  - Of acceptable quality
  - Within acceptable milestones
- By following the process, we will be able to know precisely what/how/when/where it happened !

Software Processes

Software Process is an overloaded term

- **Metaprocess:** an organization's policies, procedures, and practices for pursuing a software-intensive line of business; the focus is on organizational economics, and long-term strategies.
- **Macroprocess:** the project's policies, procedures, and practices for producing a complete software product within certain cost, schedule, and quality constraints.
- **Microprocess:** a project team's policies, procedures, and practices for achieving an artifact of the software process.

Core project management activities

- Identify requirements
- Address the various needs, concerns, and expectations of the stakeholders in the project
- Establish active, effective, collaborative communications among the stakeholders
- Manage stakeholders towards meeting project requirements and creating project deliverables
- Balance the competing project constraints, including: scope, quality, schedule, budget, resources, and risks

Software Project Trade-offs

What is the goal?

- Balance the main three in order to:
  - Stay within the budget (cost)
  - Deliver on time to gain market share (time)
  - Exceed customer satisfaction (quality)

The time/scheduling hypocrisy

- What can you tell me about the next project, you ask?
  - **It is due on June 1st tells your manager**
- We hold deadlines too dearly. Of course, time to market is critical
- But what generally happens on projects when you hit that deadline?

What makes a project successful?

Successful project management means meeting all three goals (scope, time, cost) – **and** *satisfying the project's sponsor*.

Trade-off Triangle

- **Project constraints:** Fast, cheap, good. Choose two. Also stated as: "On-time, on-budget, high-quality. Choose two."
  - **Reality often ignored in project planning**
- Know which of these are fixed & variable for every project
  - Time
  - Cost
  - Scope



Success Metrics

1. On schedule
   - Requires good: plan; estimation; control
2. Within budget
   - Again: planning, estimation & control
3. According to requirements
   - Importance of good requirements
   - Perception & negotiation critical
4. High quality. May or may not be same as item 3

Only real measure:

Is the customer happy?

Customer satisfaction!!

Factors in Project Success & Failure

Software Crisis

- Many software-related failures: auto-pilot systems, air traffic control systems, banking systems, IRS (Interface Requirements Systems).
  - On January 15, 1990, the AT&T long-distance telephone network broke down, interrupting long-distance telephone services in US for over 8 hours. [Missing **`break`** in a **`switch`** statement.]
  - On June 4, 1996, the maiden flight of the new and improved Ariane 5 rocket exploded 37 seconds after lift-off.
  - On June 8, 2001, a software problem caused the NYSE to shut down the entire trading floor for over an hour.
  - Many, many, many more.

What is the problem?

Software Projects have a terrible track record

A 1995 Standish Group study (CHAOS) found that only 16.2% of IT projects were successful in meeting scope, time, and cost goals (on-time & on-budget) [Things have improved a bit since.]

Over 31% of IT projects were canceled [never seeing completion], costing over $81 billion in the U.S. alone

- They never worked
- Too late for the market window

Most projects are

- Late in delivery
- Missing functionality
- Have major defects (bugs)
- Did not do what the customer wanted
- Hard to maintain and support

The Standish Group's CHAOS Report (2001)

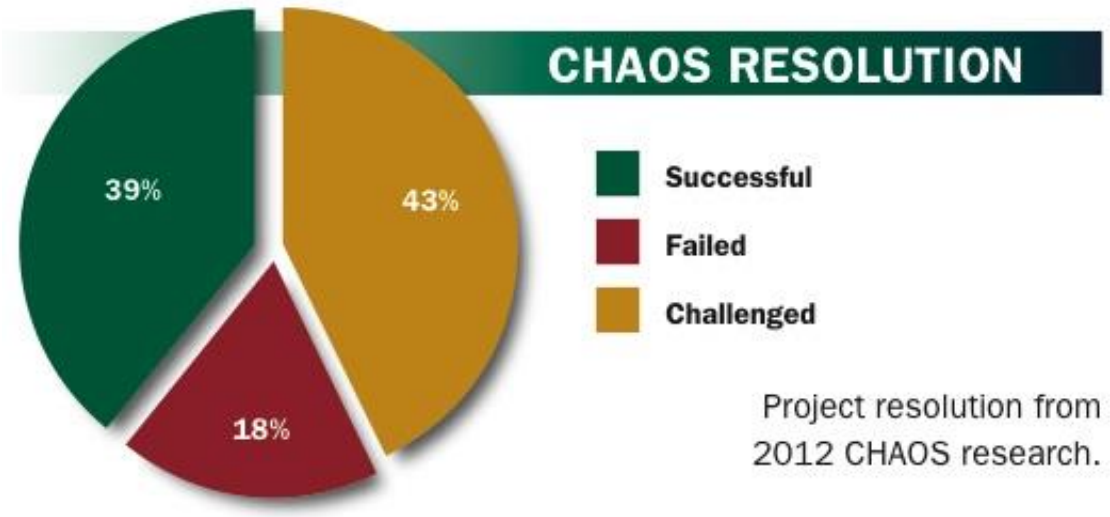
- Standish compiles and publishes a periodic survey on the success and failure rates of IT projects
- These statistics date from **2001**–however, based on more recent complementary reports, these numbers have remained fairly stable over the years:
  - Average IT project schedule overrun: 163%
  - Average IT project cost overrun: 145%
  - Actual deliverable functionality compared to plan: 67%
  - IT projects judged a success: 26%
  - Lost value from marginal and failed projects: $75 billion

Chaos Report 2012

Project Success: Type 1. The project is completed on-time and on-budget, with all features and functions as initially specified. **(2012: 39%)**

Project Challenged: Type 2. The project is completed and operational but over-budget, over the time estimate, and offers fewer features and functions than originally specified. **(2012: 43%)**

Project Impaired:
Type 3.
The project is canceled at some point during the development cycle. **(2012: 18%)** (Are ALL impaired projects failures???)



What Went Right? – Improved Project Performance

- The Standish Group's CHAOS studies show improvements in IT projects in the past decade

RESOLUTION

	2004	2006	2008	2010	2012
Successful	29%	35%	32%	37%	39%
Failed	18%	19%	24%	21%	18%
Challenged	53%	46%	44%	42%	43%

Project resolution results from CHAOS research for years 2004 to 2012.

Why the Improvements?

"The reasons for the increase in successful projects vary. First, the average cost of a project has been more than cut in half. Better tools have been created to monitor and control progress and **better skilled project managers with better management processes** are being used. The fact that there are processes is significant in itself.” *

*The Standish Group, "CHAOS 2001: A Recipe for Success" (2001).

A Case Study
The Initial Launch of HealthCare.gov (2013)



ACA – HealthCare.gov

- ACA (**Affordable Care Act**) signed into law on March 23, 2010
- **HealthCare.gov** is a healthcare exchange website.
  - "One-stop shopping sites for health insurance"
  - CBO (Congressional Budget Office) forecast: 7 million users during the first year
- Development contracts awarded in September 2011
  - No-bid, cost-plus contracts
  - Pre-certified private contractors
- HealthCare.gov launched on October 1, 2013
  - Serious technological problems

HealthCare.gov – The Launch Problems

- Performance: response time (landing page) > 8s
  - “Annoyingly long wait times"
- Navigation: broken UI
- Stability: intermittent crashes, availability ≈ 43%
- Functionality: incorrect and incomplete data
- Error rate (per page) ≈ 6%
- Scalability: < 1,100 concurrent users
- Enrollment completion rate < 30%

HealthCare.gov – The Contractors & The Cost

- The lead contractor: CGI Group
  - At least 47 private companies involved
  - Including QSSI, Equifax, Serco
- Coordinated by the Centers for Medicare and Medicaid Services (CMS)
- Total budget: $293 million
  - CGI: $196 million (2013). $112 million paid Oct. 2013
  - QSSI: $85 million
- Estimated actual cost: > $500 million by Oct. 2013

HealthCare.gov – The Failures – Software Eng.

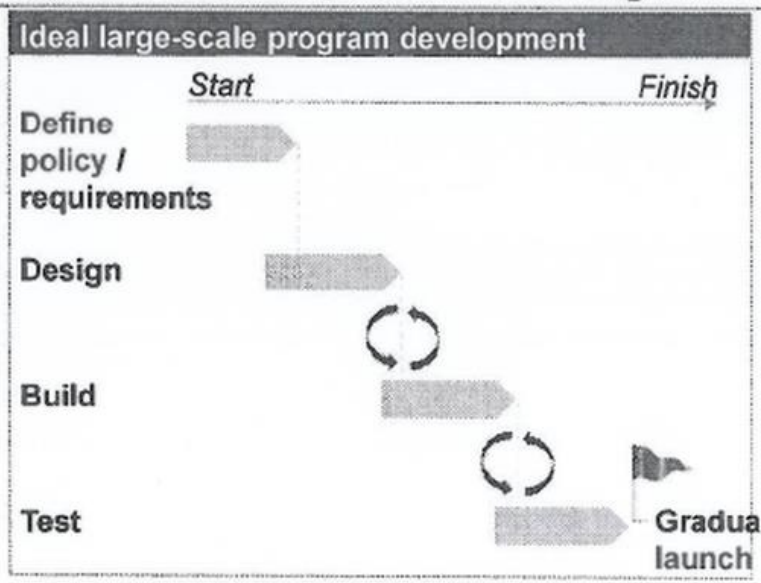
- **Inadequate Testing**
  - “This system just wasn't tested enough.” – CMS
  - Full test began T -2 weeks (time before launch).
  - Final “pre-flight checklist” T -1 week: 41 of 91 functions fail.
  - No “end-to-end” test as late as T -4 days
  - Stress tests T -1 day: performance degradation with only 1,100 concurrent users. (50,000-60,000 expected)
  - Final top-to-bottom security tests not finished.
  - No integration test. No beta test.

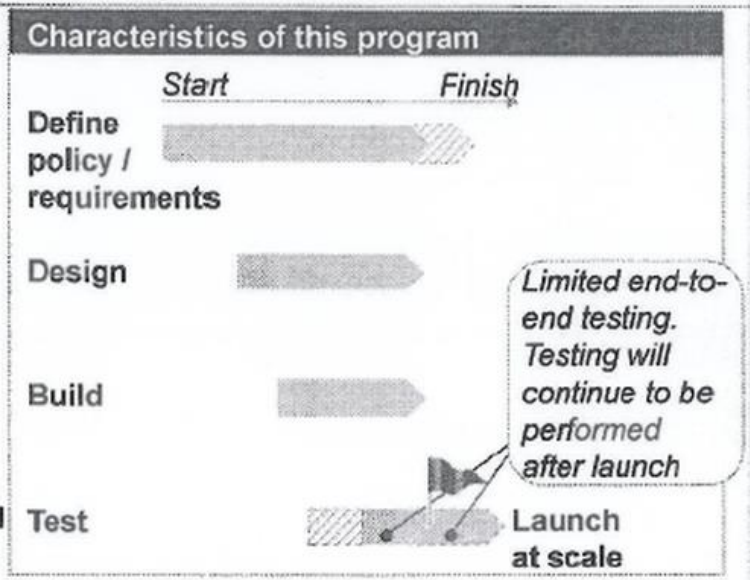
HealthCare.gov – The Failures – Software Eng.

- **Evolving, Rolling Requirements**
  - Regulations and policies were still in flux when contracts awarded in 2011.
  - The specifications for the project were delayed repeatedly.
  - The regulations and policies were modified repeatedly until summer 2013.
  - Repeated changes result in design changes.
  - CGI (Common Gateway Interface) did not start coding until Spring 2013
- **Failure to Effectively Manage Changes**
  - "Write-down-all-the-requirements-then-build-to-those-requirements"
  - Did not adopt an agile development approach.
  - Committed to an all-or-nothing launch date.

Case Study: HealthCare.gov– McKinsey "Red Team" Assessment

Programs of this type ideally have a sequential planning, design, and implementation process with significant testing and revision





Description of ideal situation:

- Clear articulation of requirements & success metrics
- Minimized dependency on third parties
- Sequential requirements, design, build, and testing
- Iteration and revision between phases
- End-to-end integrated operations and IT testing
- Limited initial launch

Current situation:

- Evolving requirements
- Multiple definitions of success
- Significant dependency on external parties/ contractors
- Parallel "stacking" of all phases
- Insufficient time and scope of end-to-end testing
- Launch at full volume

CMS has been working to mitigate challenges resulting from program characteristics

CMS

Confidential and Proprietary — Pre-decisional Information

Case Study: HealthCare.gov: The Failures – Management

- Management expertise is in getting contracts not delivering projects.
- Project quality is sacrificed for the sake of appearances.
- Seriously substandard staffing and under staffing
  - CGI. Three months before launch, only 10 developers were working on a crucial part of the site, and of those, only one was "at a high enough skill level."
- Lack of coordination among contractors
  - Unclear responsibilities. Fragmented authority.

Case Study: HealthCare.gov: The Failures – Gov. & Policies

- Government IT projects
  - Most are over budget and/or behind schedule.
  - “Write-down-all-the-requirements-then-build-to-those-requirements” is outdated.
- IT procurement policies
  - Cost-plus contract, no-bid contracts
  - “The firms that typically get contracts are the firms that are good at getting contracts, not typically good at executing on them.”

Case Study: HealthCare.gov: The Lessons Learned

- Adopt software engineering best practices
  - Agile software development. Testing early.
  - Software quality assurance and testing. Testing throughout.
- Adopt management best practices
  - Clear responsibility and accountability
  - Performance metrics and progress tracking
- Revamp government IT procurement policies
  - Current system is antiquated, and has failed.
  - Bring government IT to the 21th century.

Case Study: HealthCare.Gov – Dec. 2013

- 400+ bug fixes, by the end of Nov. 2013
  - "Operate smoothly for most users." – W.H.
- Availability > 90%
- Response time (landing page) < 1s
- Error rate (per page) < 1%
- Completion rate ≈ 80%
- System capacity ≈ 50,000 concurrent users
- Sign-ups
  - 27,000 in Oct, 110,000 in Nov, 975,000 in Dec

Case Study: HealthCare.Gov – April 2014

- Issues on data accuracy and completeness
  - Estimated 10-15% of sign-ups missing: outbound 834
  - Other inaccuracies have been reported
- CGI work during repair continue to be substandard
  - Half of the software fixes failed. – *CMS*
- CGI contract has been terminated. (Jan. 10 , 2014)
- March 31, 2014 (last day to sign up), site went down
- April 1, 2014. 7.1 million signed up – *W.H.*

Case Study: HealthCare.Gov – Aftermath

- *Accenture* took over in Jan. 2014 as the lead contractor for development and maintenance: $175M
- Cost of building the system – *GAO,* July 2014
- $834M through Feb. 2014
  - Total estimated cost: > $2B
- "CMS undertook the development of HealthCare.gov without effective planning or oversight practices"
- Also found "increased and unnecessary risk of unauthorized access, use, disclosure, modification or loss" of information
- CMS only withheld $267,000 in requested fees, 2% of the contract, from CGI.

Case Study: HealthCare.Gov – 2015 Enrollment Cycle

- Enrollment for 2015 (37 states)
  - Nov 15, 2014 – February 15, 2015.
  - Outages on the first day
  - More smooth operation thereafter
- February, 2015
  - ~ 11.4 million sign-ups (~8.6 million re-enrollments)
    - Open enrollment extensions: March 15 – April 30
  - ~800,000 received incorrect tax information,
    - Incorrect amount on 1095-A Form

What is the problem?

Ever-Present Difficulties

- Few guiding scientific principles
- Few universally applicable methods
- As much people problems as technological
  - managerial / psychological / sociological
- Sponsors unwilling to spend money for supposedly unrewarding activities
  - Quality
- Organizational rivalries
- Time pressure
- Cost pressure

Why do IT projects fail?

- Requirements, requirements, requirements
  - Requirements are unclear, incomplete, or the project management methodology does not accommodate changing requirements effectively
- Lack of user involvement
  - A disengaged or absentee user (~customer) is an invitation for project problems or outright failure
  - Often the result of a lack of business and IT integration
- Unrealistic expectations (including unrealistic time frame expectations)
  - Closely related to/result of ‘Lack of user involvement’
  - “How hard can it be?” attitude of business and technical management
  - Overly optimistic ‘can do’ attitude at all levels
  - It is the project management team's responsibility to educate users in the realities of the project

Why do IT projects fail?

- Lack of planning
  - You cannot achieve any goal without planning
  - Proper approach to planning in a complex project must be adaptive: big planning up front is a waste of time and gives a false sense of security
  - Related issues: Perceived rush to get started; overconfidence
- Unclear (or lack of) vision and/or objectives
  - The project vision must be established as one of the first steps in project planning
  - The vision for a project should be short, concise, and laser-sharp: the ‘elevator statement’ format is most effective

Why do IT projects fail?

36 Classic Mistakes

- AntiPatterns
  - Seductive Appeal: good reason for decisions at the time
- Types
  - People-Related
  - Process-Related
  - Product-Related
  - Technology-Related
- Gilligan's Island

People-Related Mistakes Part 1

- Undermined motivation
- Weak personnel
  - Weak vs. Junior
- Uncontrolled problem employees
- Heroics
- Adding people to a late project
- Lack of match between people and needs
  - Incompetent or over competent
  - Culture clash

People-Related Mistakes Part 2

- Noisy, crowded offices
- Customer-Developer friction
  - Unrealistic expectations
  - Lack of user input
- Politics over substance
- Lack of effective project sponsorship
- Lack of stakeholder buy-in
- Wishful thinking

Process-Related Mistakes Part 1

- Optimistic schedules
  - Omitting necessary tasks from estimates
  - Planning to catch-up later
  - Code-like-hell programming
  - Insufficient risk management
- Contractor failure
- Insufficient planning
- Abandonment of plan under pressure

Process-Related Mistakes Part 2

- Wasted time during fuzzy front end
- Shortchanged upstream activities
- Inadequate design
- Shortchanged quality assurance
- Insufficient management controls
- Frequent convergence

Product-Related Mistakes

- Requirements gold-plating
  - Gilding the lily
- Feature creep
- Developer gold-plating
  - Beware the pet project
- Push-me, pull-me negotiation
  - Slip schedule + add features
- Research-oriented development

Technology-Related Mistakes

- Silver-bullet syndrome
- Overestimated savings from new tools and methods
  - Fad warning
- Switching tools in mid-project
- Lack of automated source-code control

What Helps Projects Succeed?*

1. Executive support
2. User involvement
3. Experienced project manager
4. Clear business objectives
5. Minimized scope
6. Standard software infrastructure
7. Firm basic requirements
8. Formal methodology
9. Reliable estimates
10. Other criteria, such as small milestones, proper planning, competent staff, and ownership

*The Standish Group, “Extreme CHAOS,” (2001).

Four Project Dimensions

Four Project Dimensions (The 4 P's)

- **People** — the most important element of a successful project
- **Product** — the software to be built
- **Process** — the set of framework activities and software engineering tasks to get the job done
- **Project** — all work required to make the product a reality

People

"It's always a people problem"

– Gerald Weinberg, "The Secrets of Consulting"

- Developer productivity: 10-to-1 range
  - Teams 3 (or 5) to 1 difference
- Improvements:
  - Team selection
  - Team organization
  - Motivation
- Other success factors
  - Matching people to tasks
  - Career development
  - Balance: individual and team
  - Clear communication

Process

- Is process stifling?
  - 2 Types: Management & Technical
- Development fundamentals
- Quality assurance
- Risk management
- Lifecycle planning
- Avoid abuse by neglect
- Customer orientation
- Process maturity improvement
- Rework avoidance
- **Goals**
  - cut time-to-market
  - Improve quality

Product

- The “tangible” dimension
- Product size management
- Product characteristics and requirements
- Feature creep management

Technology

- Often the least important dimension
- Language and tool selection
- Value and cost of reuse

Planning

- Determine requirements
- Determine resources
- Select lifecycle model
- Determine product features strategy
- **Tracking**
  - Cost, effort, schedule
  - Planned vs. Actual
  - How to handle when things go off plan?

Measurements

- To date and projected
  - Cost
  - Schedule
  - Effort
  - Product features
- Alternatives
  - Earned value analysis
  - Defect rates
  - Productivity (ex: SLOC)
  - Complexity (ex: function points)

Technical Fundamentals

Assumed Software Development Life Cycle (SDLC)

- Requirements
- Analysis
- Design
- Construction
- Quality Assurance (aka Testing)
- Deployment

Software Project Management Fundamentals

Formal Project Management

Advantages of Using Formal Project Management

- Better control of financial, physical, and human resources
- Improved customer relations
- Shorter development times
- Lower costs
- Higher quality and increased reliability
- Higher profit margins
- Improved productivity
- Better internal coordination
- Higher worker morale (less stress)
  - Less “death marches”
  - Less overworked personnel

Getting organized

So, … now what?

- Who is involved?
  - Stakeholders
- What do they want done?
  - Charter, vision, requirements
- Who do we have available to do the work?
  - Resources and staffing
- How do we do this?
  - Project planning, WBS
- How much will it cost
  - Estimating
- When will it be finished?
  - Scheduling
- What can possibly go wrong?
  - Risk Management

Summary of essential points

- Projects and operations have both similarities and differences
- Complex projects exhibit highly contingent behavior, are unpredictable, and face complex risks
- Virtually all IT projects should be considered complex projects
- Complex project management requires integration of significant sociological, psychological, and technical skills
- The factors contributing to project success and failure are reasonably well-known
- This course discusses ways to achieve project success factors

Software Project Management

Fundamentals

Formal Project Management

Advantages of Using Formal Project Management

- Better control of financial, physical, and human resources
- Improved customer relations
- Shorter development times
- Lower costs
- Higher quality and increased reliability
- Higher profit margins
- Improved productivity
- Better internal coordination
- Higher worker morale (less stress)
  - Less “death marches”
  - Less overworked personnel

What Helps Projects Succeed?*

1. Executive support
2. User involvement
3. Experienced project manager
4. Clear business objectives
5. Minimized scope
6. Standard software infrastructure
7. Firm basic requirements
8. Formal methodology
9. Reliable estimates
10. Other criteria, such as small milestones, proper planning, competent staff, and ownership

*The Standish Group, “Extreme CHAOS,” (2001).

Conventional Software Management Performance

Barry Boehm's “Industrial Software Metrics Top 10 List”:

1. Finding and fixing a software problem after delivery costs 100 times more than finding and fixing the problem in early design phases
2. You can compress software development schedules 25%, but no more
3. For every $1 you spend on development, you will spend $2 on maintenance
4. Software development and maintenance costs are primarily a function of source lines of code.
5. Variations among people account for the biggest difference in software productivity; hire good people to succeed.

Conventional Software Management Performance

Barry Boehm's “Industrial Software Metrics Top 10 List”:

6. The overall ratio of software to hardware costs is still growing.
7. Only about 15% of software development effort is devoted to programming
8. Software systems and products typically cost 3 times as much per SLOC as individual software programs. Software system products (system of systems) costs 9 times as much
9. Walkthroughs catch 60% of the errors
10. 80% of the contributions comes from 20% of the contributors.

First Principles

- One size does not fit all
- Spectrums
  - Project types
  - Sizes
  - Formality and accuracy

Strategy

Hope is not a strategy.

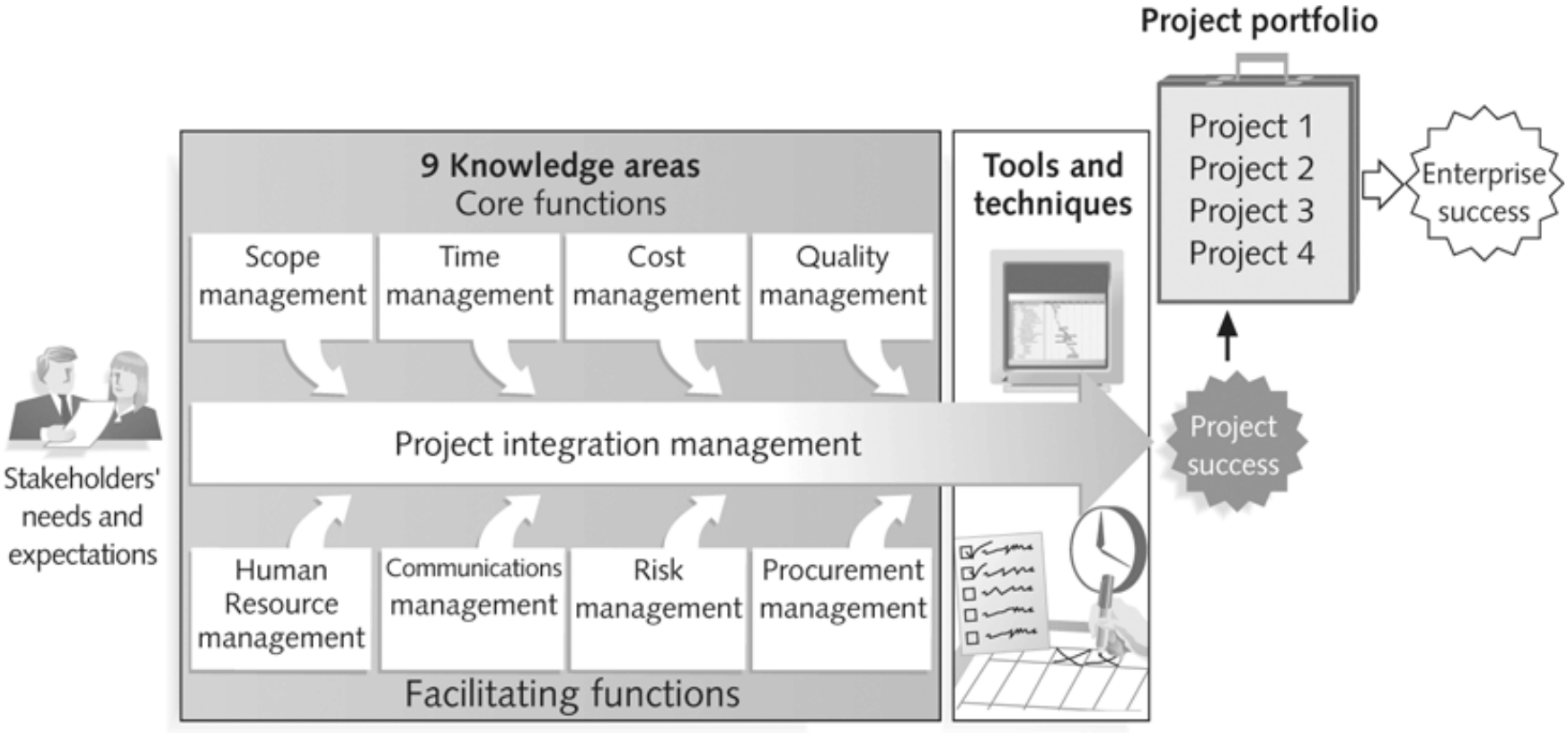
So what is our strategy?

- Classic Mistake Avoidance
- Development Fundamentals
- Risk Management
- Schedule-Oriented Practices

PMI's 9 Knowledge Areas

- Project ***integration*** management
- Scope
- Time
- Cost
- Quality
- Human resource
- Communications
- Risk
- Procurement

Project Management Framework



What is a project life cycle?

- The project life cycle is a collection of sequential or overlapping project phases
  - The phases divide the project into logical blocks of related activities
  - This division into phases simplifies management, planning, and control
- Phases within the project are defined by technical information transfer or technical component hand-off
  - Example: Inception and elaboration phases in the Unified Process
  - Example: Releases in Agile life cycles

Phases

- The completion and approval of one or more *deliverables* (defined as measurable, verifiable work products) defines the endpoint of a project phase
- Different phases can have different relationships among themselves, even within the same project
  - *Sequential relationship*. A phase starts only when the previous phase is complete
  - *Overlapping relationship*. A new phase can be planned and started before the previous phase is complete

PMBOK project life cycles

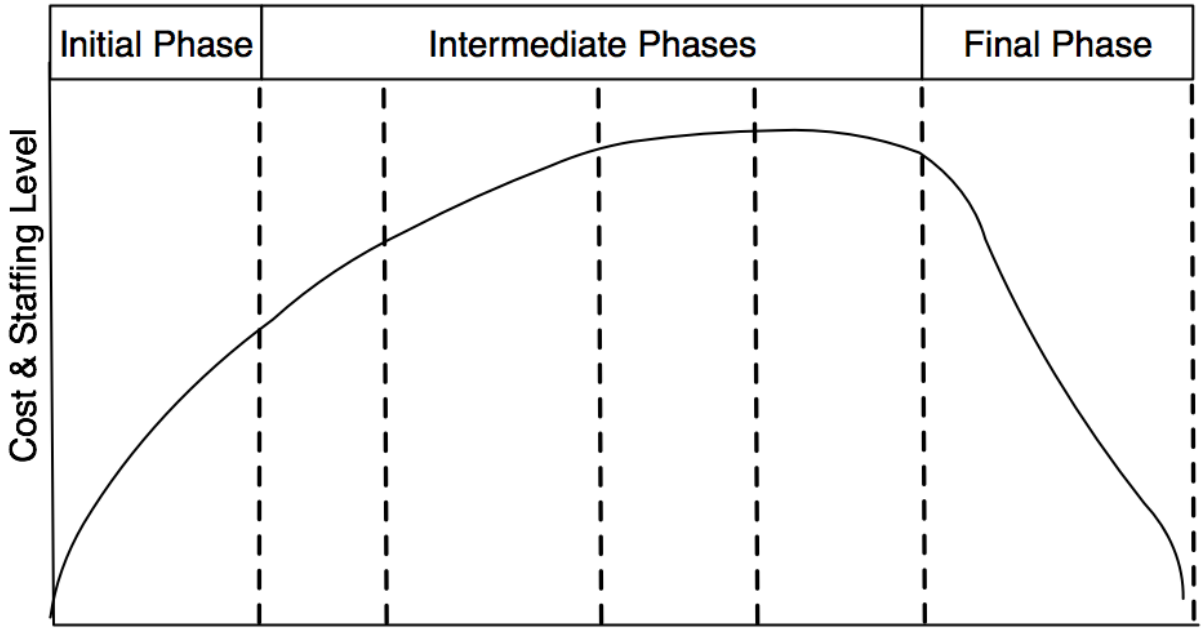
- In a *predictive life cycle*:
  - Product and deliverables are defined at the beginning of the project
  - Changes to scope are carefully–and restrictively–managed
- In an *iterative and incremental life cycle*:
  - Project phases repeat one or more project activities, taking advantage of increased understanding of the product
  - Each phase (and each iteration within a phase) successively adds to the functionality of the product
  - Scope is usually well-defined early in the project life cycle, but can be changed with relatively low overhead as project proceeds
- In an *adaptive life cycle [Agile]*:
  - Product is developed over multiple phases, each with several iterations
  - Detailed scope is defined for each phase only as the phase begins

IT project life cycles

- IT projects have two concurrent life cycles:
  - *Project life* cycle (PLC) encompasses all activities of project, including the System/Software Development Life Cycle (SDLC)
  - PLC is directed toward achieving *project* requirements
  - SDLC is directed toward achieving *product* requirements
- Both life cycle models are needed to manage an IT project
  - PLC alone will not adequately address system development concerns
  - SDLC alone will not adequately address business and product integration concerns
  - Effective integration of the two life cycle models is essential to improving the likelihood of project success
- In effect, the PLC and the SDLC should be so closely interwoven that they need not be distinguished from each other

What is a project life cycle?

- Consists of a number of generally sequential phases
- Phases are defined by technical information transfer or technical component hand-off
- Cost and staffing levels vary as a function of time according to the following qualitative schematic diagram:



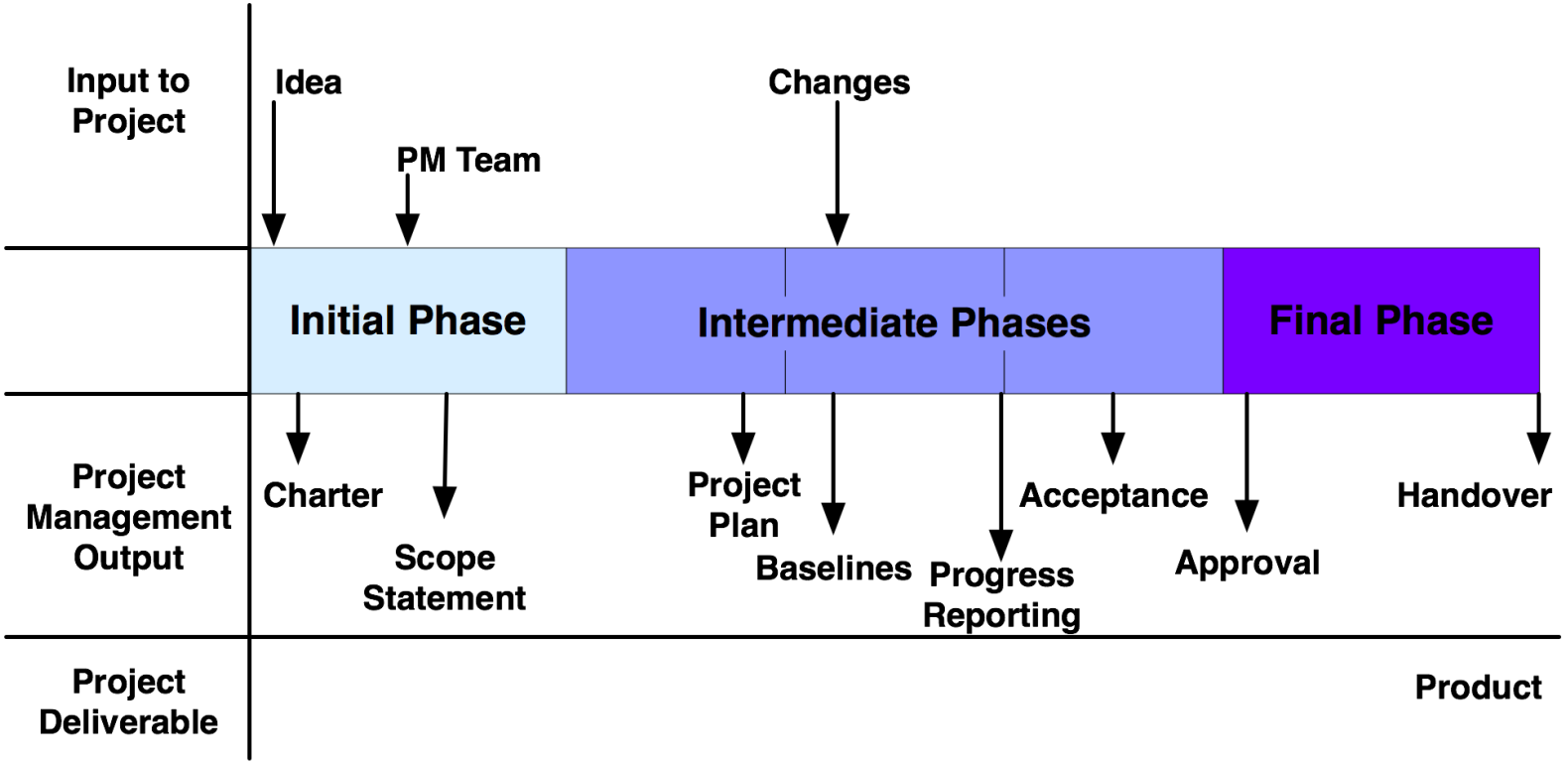
Adapted from PMBOK Guide 3rd Edition Fig. 2-1

What is a project life cycle?

- Risk of failure is greatest at start of project when the level of uncertainty is highest
- Stakeholder influence over project /product decreases as project continues
- Project life cycles define:
  - Technical work to be done in each phase
  - When deliverables are to be generated in each phase
  - How each deliverable is reviewed, verified, and validated
  - Who is involved in each phase
  - How to control each phase
  - How to approve each phase

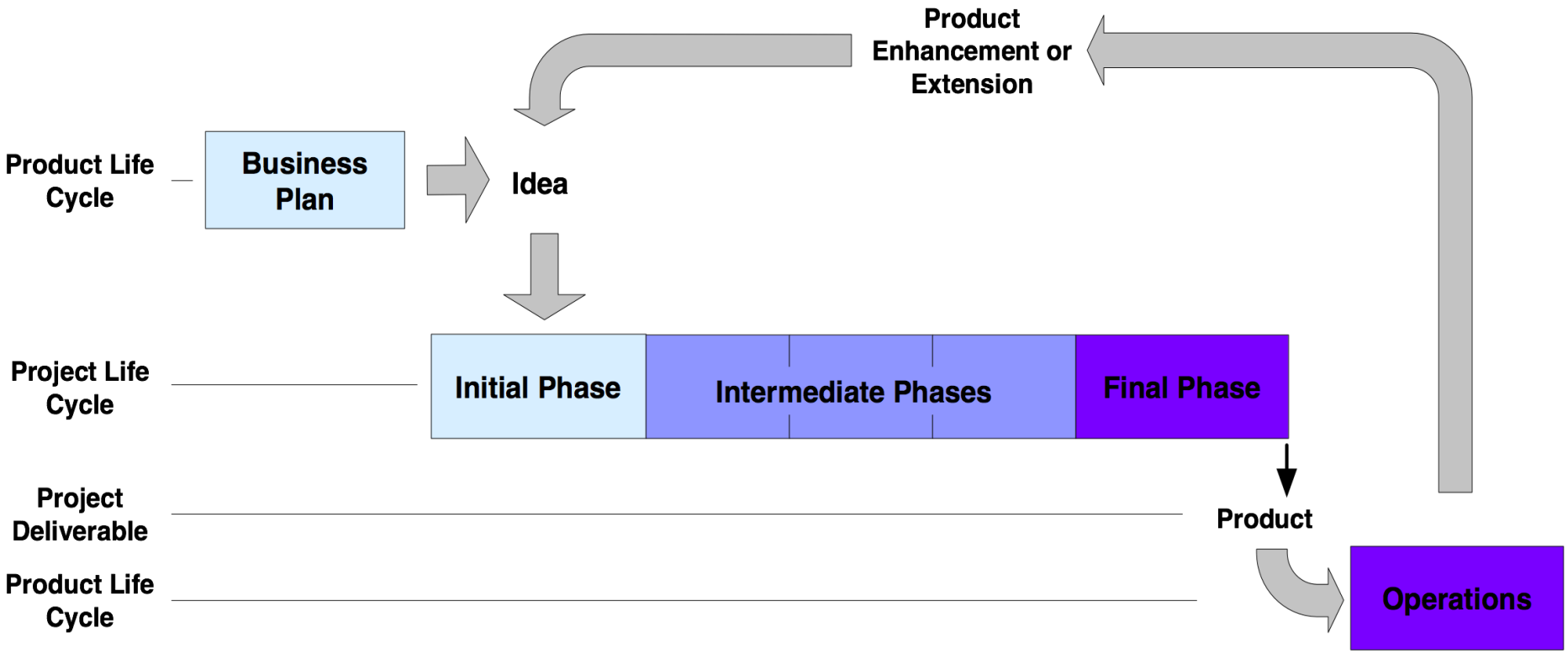
Phases in project life cycle

- The completion and approval of one or more deliverables (measurable, verifiable work product) defines a project phase
- In **iterative systems development**, new phase can be started without closing the previous phase
- A phase can be closed without initiating subsequent phase



Adapted from PMBOK Guide 3rd Edition Fig. 2-3

Project & product life cycles



Adapted from PMBOK Guide 3rd Edition Fig. 2-4

The systems development lifecycle

- "The *systems development life cycle (SDLC) is the process of* understanding how an information system (IS) can support business needs by designing a system, building it, and delivering it to users"*
- A *methodology is a formalized approach to implementing the SDLC*
- What differentiates one methodology from another:
  - The specific activities that must be performed
  - When, how, and how often the activities are performed
  - Who performs the activities
  - The amount of emphasis placed on an activity at a specific point in time

* Dennis, Alan (2012-05-01). *Systems Analysis and Design with UML, 4th Edition (Page 2). Wiley. Kindle Edition.*

Software Development Process

- Prescriptive
  - Linear/sequential (Classic and Waterfall)
  - Evolutionary (Iterative/incremental or spiral)
  - Unified Process (Unified Process is **based on the enlargement and refinement of a system through multiple iterations**, with cyclic feedback and adaptation.)
- Adaptive
  - Lean and agile methods

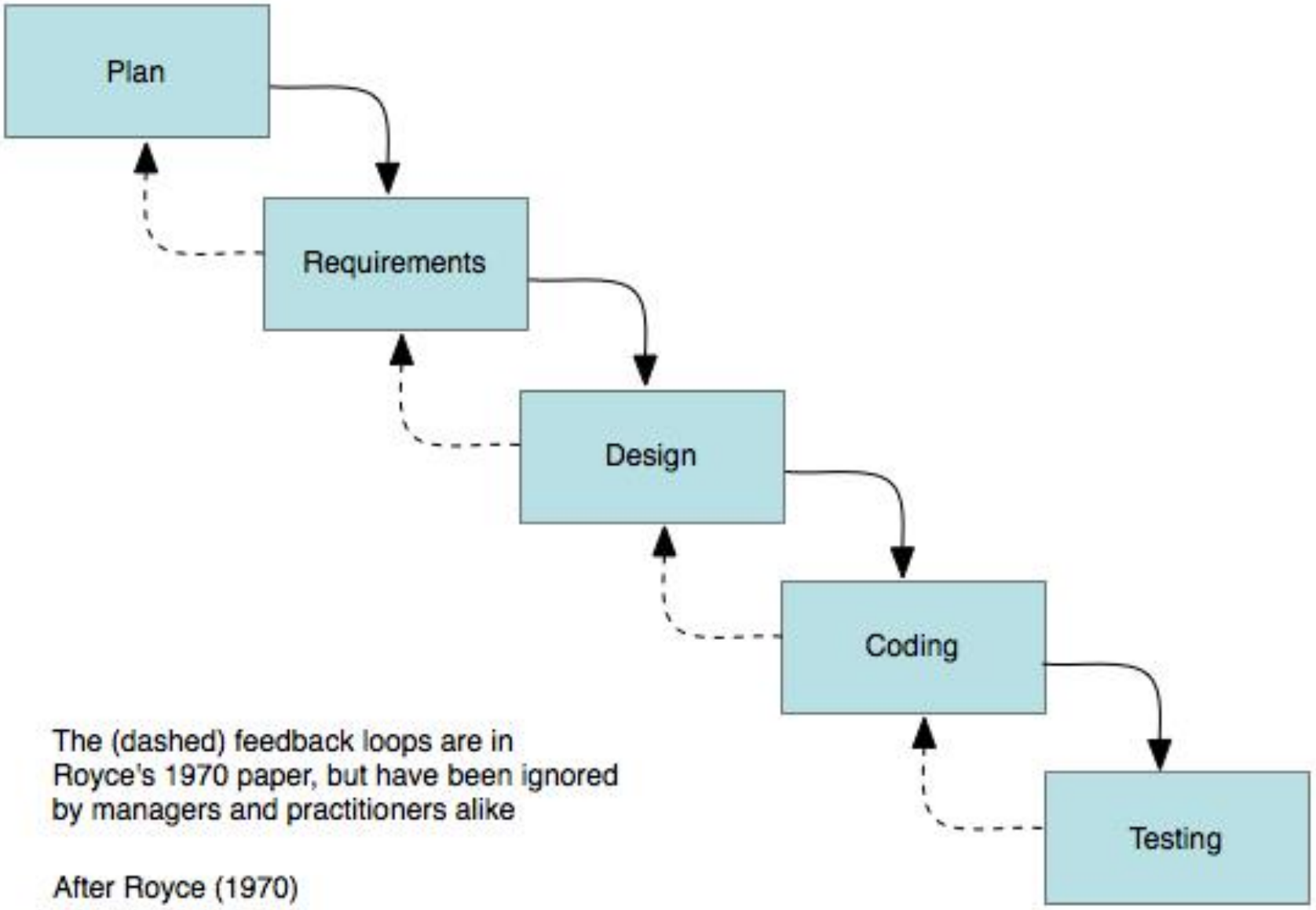
Sequential (‘waterfall’) methodology

- The term *waterfall* was coined by Winston Royce in a 1970 paper titled *Managing the Development of Large Software Systems*, in the Proceedings of IEEE WESCON
- The paper used the sequential waterfall approach as an example of an ***ill-conceived, risk-prone practice*** for developing large systems
- Royce advocated a series of iterative feedback loops among the development stages, incrementally gaining learning value from working software
- Instead of adopting the approach Royce advocated, managers and practitioners adopted its *anti-form, without* feedback loops

Waterfall SDLC

- Each phase is marked by completion of Deliverables
- The primary software project phases:
  - Requirements
  - Analysis
  - Design
  - Construction
  - Quality Assurance (aka Testing)
  - Deployment

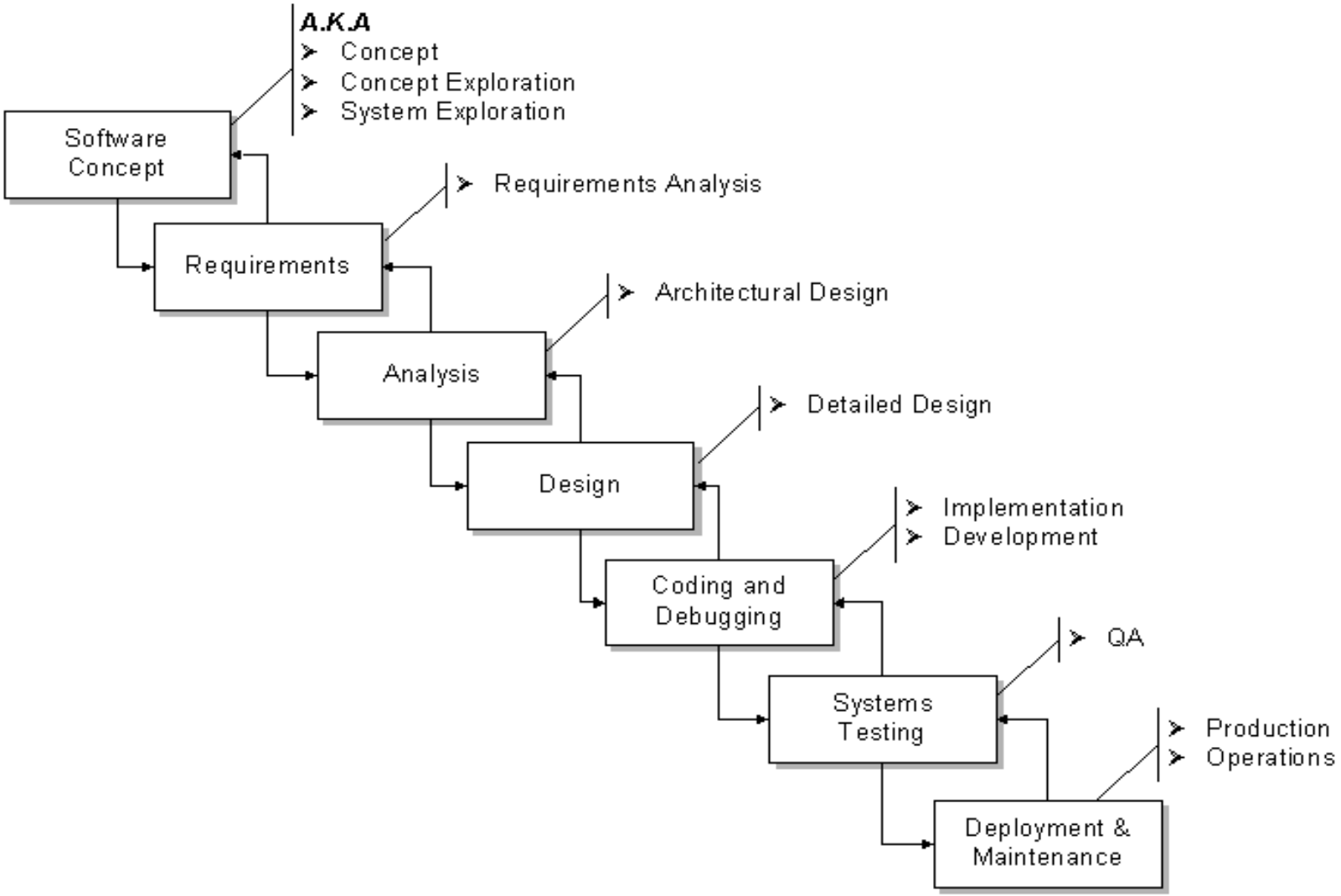
Waterfall SDLC



The (dashed) feedback loops are in Royce's 1970 paper, but have been ignored by managers and practitioners alike

After Royce (1970)

Project Phases A.K.A.



Waterfall system development model

- Highly-sequential process
- Failure symptoms:
  - Extended integration and late design breakage
  - Late risk resolution
  - Requirements-driven functional decomposition
  - Argumentative stakeholder relationships
  - Focus on documents and review meetings
- Still followed (in name or practice) by many organizations, usually a modified version

Waterfall system development model

Sequential: suitable projects and management approaches

- A sequential SDLC is suitable for projects with:
  - Clear, unambiguous, and stable user requirements
  - Familiar, proven technology
  - Low complexity
  - Adequate time
  - Stable schedule
- A project meeting most of these criteria can use conventional project management practices, such a big, up-front planning and conventional risk assessment

Evolutionary methodologies

- An *evolutionary methodology* follows an iterative and incremental approach that allows the start of development with incomplete, imperfect knowledge
- An iterative and incremental process is like solving a jigsaw puzzle: neither top-down nor bottom-up but **buildup** and **converge**
- An iterative and incremental process offers these advantages:
  - Logical progress toward evolving a robust architecture
  - Effective management of changing requirements
  - Effective means to address changes in planning
  - Ability to perform continuous integration
  - Early understanding of the system (the ‘Hello world!’ effect)
  - Ongoing risk assessment
- Evolutionary methodologies are incremental at both the macro (project-scale) and micro (working team) process levels

Iterative system development model

- Non-linear approach to system development
- Incorporates top five principles of modern development processes:
  - *Architecture first*. Provides the central design element
  - *Iterative life-cycle process*. Provides the essential risk management element
  - *Component-based development*. Provides the technology element
  - *Change management environment.* Provides the control element
  - *Round-trip engineering*. Provides the automation element

5,000 foot view of Iterative SDLC

- **Iterative** SD model defines four life-cycle phases:
  - Inception
  - Elaboration
  - Construction
  - Transition
- We **iterate** through each phase, and repeat as needed.
- Now, for a quick survey of the phases…

Inception phase

- Essential activities
  - *Formulate product scope*. Capture requirements and operational concept
  - *Perform feasibility analysis*. Determine whether the organization has the resources and technical capabilities to meet customer's needs
  - *Synthesize the system architecture*. Evaluate essential system design constraints and trade-offs, as well as available solutions
  - *Plan and prepare business case*. Address risk management, staffing, iteration plans, cost, and infrastructure

Elaboration phase

- Most critical phase of the four
- Essential activities
  - *Elaborate the vision*. Detail elements of the vision that drive architectural or planning decisions
  - *Elaborate the process and infrastructure*. The construction process and environment are established here
  - *Elaborate the architecture and select reusable (internal or COTS) components*. Baseline the architecture as quickly as possible and demonstrate that the architecture will support the vision at reasonable cost in reasonable time

Construction phase

- Essential activities
  - *Achieve useful versions* (intermediate, alpha, beta, and other test releases)
  - *Perform resource management, control, and process optimization*
  - *Complete component development and test*
  - *Assess product releases against acceptance criteria*

Transition phase

- Essential activities
  - *Perform deployment-specific engineering tasks*. Commercial packaging and production, sales kit development, field personnel training
  - *Assess deployment baselines against complete vision and acceptance criteria.* Examine and compare what is being delivered to what was envisioned and delineated by acceptance criteria
  - Plan for next iteration

Comparative expenditure profiles

Waterfall		Iterative	
Activity	**Cost**	**Cost**	**Activity**
Management	5%	10%	Management
Requirements	5%	10%	Requirements
Design	10%	15%	Design
Code & Unit Testing	30%	25%	Implementation
Integration & Test	40%	25%	Assessment
Deployment	5%	5%	Deployment
Environment	5%	10%	Environment
Total	100%	100%	Total

Based on and adapted from Tables 1-1 and 10-1 in
Software Project Management: A Unified Approach by Walker Royce

Suitable Projects And Management Approaches

- An evolutionary SDLC is suitable for projects with:
  - Reasonably–but not perfectly–clear user requirements
  - Unfamiliar or unproven technology
  - High complexity
  - Short time schedule
  - Schedule variability
- Such a project would use rolling wave planning rather than big, up-front planning and use a continuous, adaptive approach to risk assessment and management

Agile Project Management

Agile Projects

- Lean methodology. Only as much process as necessary.
- 'Agile' is an umbrella term used for identifying various models used for agile development, such as Scrum.
- Scrum is **an agile project management framework** that helps teams structure and manage their work through a set of values, principles, and practices
- Since agile development model is different from conventional models, agile project management is a specialized area in project management.

Agile Projects

- Agile project management is an iterative approach to planning and guiding project processes.
- An agile project is completed in small sections called iterations, or, in scrum, sprints.
- Each iteration is reviewed and critiqued by the project team, which may include representatives of the client business as well as employees.
- Insights gained from the critique of an iteration are used to determine what the next step should be in the project.
- Each project iteration is typically scheduled to be completed within two weeks.

Agile Project Steps

1. The **product owner** identifies the **product vision**.
2. The **product owner** creates a **product roadmap**.
3. The **product owner** creates a **release plan**.
4. The **product owner**, the (scrum) **master**, and the **development team** plan sprints, also called iterations, and start creating the product within those sprints
5. During each sprint, the development team has daily meetings [called scrums].
6. The team holds a sprint review.
7. The team holds a sprint retrospective.

Agile Project Artifacts

1. **Product vision statement:** An elevator pitch, or a quick summary, to communicate how your product supports the company's or organization's strategies. The vision statement must articulate the goals for the product. Revisit once a year.
2. **Product roadmap:** The product roadmap is a high-level view of the product requirements, with a loose time frame for when you will develop those requirements. Revisit twice a year.
3. **Release plan:** A high-level timetable for the release of working software.
4. **Product backlog:** The full list of what is in the scope for your project, ordered by priority. Once you have your first requirement, you have a product backlog.
5. **Sprint backlog:** The goal, user stories, and tasks associated with the current sprint.
6. **Increment:** The working product functionality at the end of each sprint.

Agile Project Roles

1. **Development team:** The group of people who do the work of creating a product. Programmers, testers, designers, writers, and anyone else who has a hands-on role in product development is a member of the development team.
2. **Product owner:** The person responsible for bridging the gap between the customer, business stakeholders, and the development team. The product owner is sometimes called a *customer representative.*
3. **Scrum master:** The person responsible for supporting the development team, clearing organizational roadblocks, and keeping the agile process consistent. A scrum master is sometimes called a project facilitator.
4. **Stakeholders:** Anyone with an interest in the project.
5. **Agile mentor:** Someone who has experience implementing agile projects and can share that experience with a project team. The agile mentor can provide valuable feedback and advice to new project teams and to project teams that want to perform at a higher level.

Agile Project Events

1. **Project planning:** The initial planning for your project.
   - includes creating a **product vision** statement and a **product roadmap,**
   - can take place in as little time as one day.
2. **Release planning:** Planning the next set of product features to release
3. **Sprint:** A short cycle of development, in which the team creates potentially shippable product functionality.
4. **Sprint planning:** A meeting at the beginning of each sprint where the scrum team commits to a sprint goal.
5. **Daily scrum:** A 15-minute meeting held each day in a sprint, where development team members state what they completed the day before, what they will complete on the current day, and whether they have any roadblocks.

Agile Project Events

6. **Sprint review:** A meeting at the end of each sprint, where the development team demonstrates the working product functionality it completed during the sprint.
7. **Sprint retrospective:** A meeting at the end of each sprint where the scrum team discusses what went well, what could change, and how to make any changes.

Selection considerations: guiding questions

- Organizational characteristics
  - What are the characteristics of the organizational culture? What are the management comfort levels with the various methodologies?
  - How open is management and the organization to change?
  - Is the organization risk-tolerant or risk-adverse?
  - What is the organization's tolerance for real risk vs. perceived risk?
- Project characteristics
  - How large is the project?
  - What is the project's estimated duration?
  - Are teams co-located or distributed?
  - Is regulatory compliance a significant factor?
  - How flexible are documentation requirements?

Selection considerations: guiding questions

- People and management characteristics
  - What are the experience levels of team members?
  - Are team members self-motivated or command-driven?
  - What sort of management style is employed? Laissez-faire, micromanagement, or somewhere in-between?
  - What sort of social dynamics govern project efforts within the organization? Cooperative and problem-solving, adversarial, or blaming?

Methodology characteristics compared

System Characteristics	'Waterfall'	Plan-Driven Evolutionary	Agile Evolutionary
Unclear user requirements	Poor	Good	Excellent
Unfamiliar technology	Poor	Good	Good
Complex	Fair	Very Good	Good
Reliable	Good	Good	Excellent
Short time schedule	Poor	Excellent	Excellent
Schedule variability	Poor	Excellent	Excellent

Examples: Applying the table

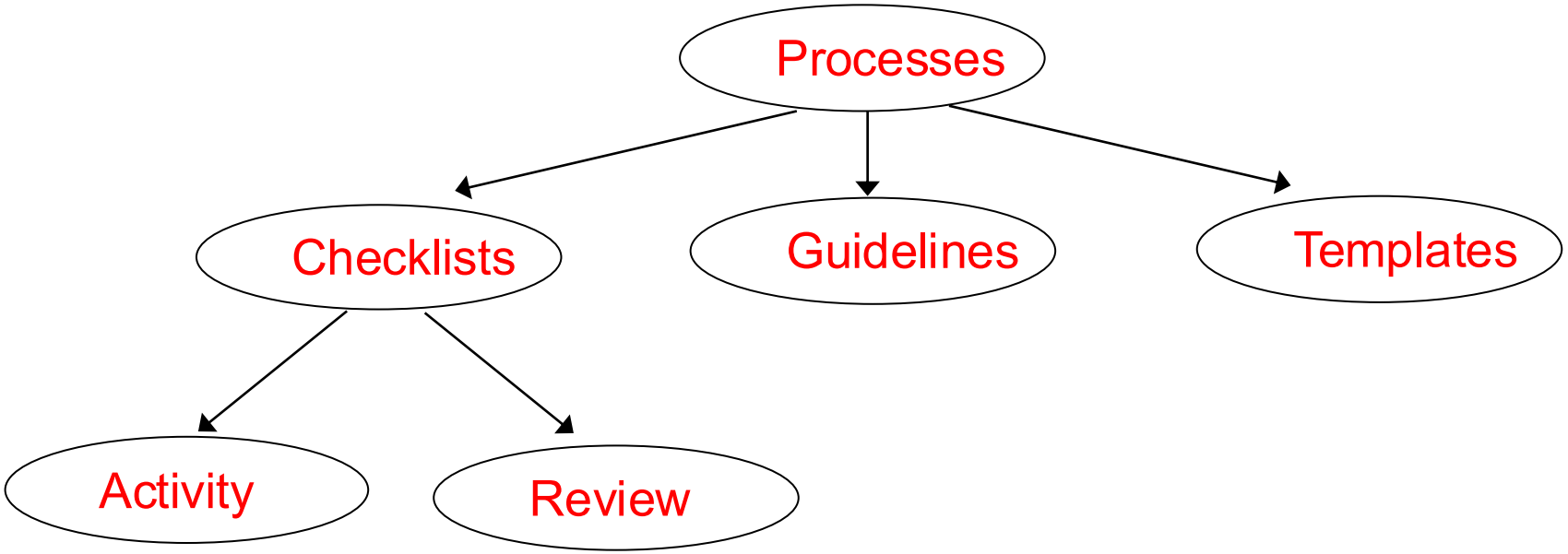
1. Short time schedule + shifting user requirements
   - Agile
2. Complex + short time schedule
   - Iterative
3. Clear user requirements + long time schedule + command-driven team
   - Water-fall
4. Reliable + complex + schedule variability
   - Agile
5. Unfamiliar technology + short time schedule + schedule variability
   - Either Agile or Iterative

Software Project Management

Process

Process

- A process encapsulates an organization's experience in form of successful recipes.
- Process descriptions, generally, contain the sequence of steps to be executed, who executes them, the entry/exit criteria for major steps, etc.
- Guidelines, checklists, and templates provide support to use the processes.



Putting a Process in Place

- Choosing a Process.
  - All projects have a process, unfortunately some don't specify and implement their process.
  - Projects with no specified process end up thrashing.
  - Thrashing, unproductive work, can quickly damage a project.
- Generally, there are two choices for choosing a process:
  1. Tailor the organizational process to your project.
     - Used when most of the people are from the same group as before
     - Used when the last project was successful.
  2. Specify a process for your project.
     - Good when people are from different organizations using different processes

Tailoring a Process

- Steps to Tailoring an Organizational Process:
  1. Determine how your project differs from the typical organizational project.
  2. Form two lists: activities your project needs from the organizational process and tasks your project doesn't need from the process
  3. Propose changes to the organizational process
  4. Circulate the tailored process within the team and other key personnel for review and input.
  5. Integrate the changes and move quickly for closure.

Assessing the Process

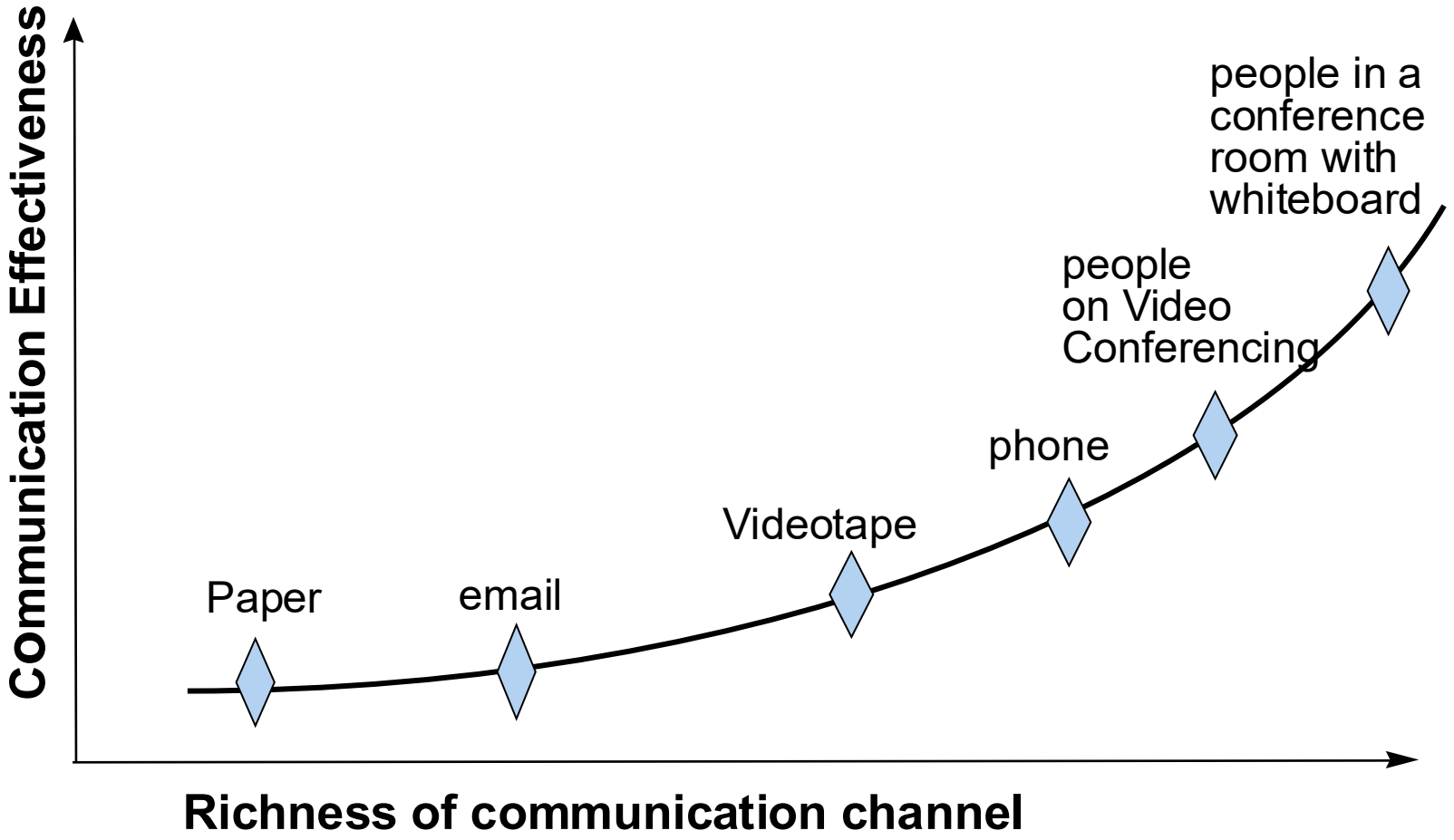
- Assessing should be an ongoing process through out the project.
- Both the project and the process should lend themselves to assessment and improvement.
- Make gathering measurements part of concurrent documentation.
- Gather data to answer the following:
  - Were the tasks and supporting activities effective?
  - How much effort did each task and activity require?
  - What tasks and activities were performed but weren't in the process specification?
  - How did the products change over time?
  - When did tasks and activities start and stop?
  - How did tasks and activities integrate?
  - When in the project did we spend effort doing what?
- Repeat this during project close out.

The Project Manager: Responsibilities

- Project planning
- Managing the project
- Lead project team
- Building client partnerships
- Targeting to the business

Few Rules Before We Embark

And finally, communicate, communicate, and communicate!



Recap

Definition of a Project

- A *project* is a sequence of unique, complex, and connected activities having one goal or purpose and that must be completed by a specific time, within budget, and according to specification.

What is a Program?

- A *program* is a collection of projects.
- The projects must be completed in a specific order for the program to be considered complete. Because they compromise multiple projects, they are larger in scope than a single project.

Project Parameters

- Five constraints operate on every project:
  - Scope
  - Quality
  - Cost
    - Time
    - Resources
- A change in one of these constraints can cause a change in another constraint to restore the equilibrium of the project
- Let's discuss each one of these in detail …

Project Parameters

Scope

- *Scope* is a statement that defines the boundaries of the project. It tells not only what will be done but also what will not be done.
- In the information systems industry, scope is often referred to as a functional specification.
- In the engineering profession, it is generally called a statement of work.

Quality

- Two types of quality are part of every project:
  - The first is ***product quality***. This refers to the quality of the deliverable form of the project.
  - The second type of quality is ***process quality***, which is the quality of the project management itself. The focus is on how well the project management process works and how can it be improved. Continuous quality improvement and process quality management are the tools used to measure process quality.

Project Parameters

Cost – The X-amount of dollars that it will cost to do the project is another variable that defines the project; the budget that has been established for the project.

- This is an important factor for projects that create deliverables that are sold to external customers

Time – The customer specifies a timeframe within which the project must be completed.

- Cost and time are inversely related to one another. The time a project takes to be completed can be reduced, but cost increases as a result.

Resources – Resources are assets, such as people, equipment, physical facilities, or inventory, that have limited availabilities, can be scheduled, or can leased from an outside party. Some are fixed, others are variable only in the long term. In any case, they are central to the scheduling of project activities and the orderly completion of the project.

Phases of the Project Management

- There are five phases of the project management life cycle:
  - **Scope/Define/Initiate** – Scope the project
  - **Plan** – Develop the project plan
  - **Execute** – Launch the plan
  - **Monitor** – Monitor/control project progress
  - **Close** – Close out the project
- Note: these can be repeated for each phase
- Each process/phase/activity is described by:
  - Inputs
  - Tools & Techniques
  - Outputs

Initiating Process

- Develop project charter
  - State the problem/opportunity.
  - Concerned with authorizing a project
  - May be used for a whole project
  - May be used for a single project phase in a large, multiphase project
- Develop preliminary project scope statement
  - Concerned with producing a preliminary, high-level definition of project
  - Broadly defines what is and what is not part of the project
- Establish the project plan.
  - Define the project objectives.
  - Identify the success criteria.
  - List assumptions, risks, obstacles

Initiating Process

- Inputs
  - Product Description
  - Strategic plan
  - Project Selection Criteria
  - Historical Information
- Outputs
  - Project Charter
  - Project Manager assigned
  - Constraints
  - Assumptions

Planning Process

Devising and maintaining a workable scheme to accomplish the business need that the project was undertaken to address

- Scope Planning
- Scope Definition
- Activity Definition
- Activity Sequencing
- Activity Duration Estimating
- Resource Planning
- Cost Estimating
- Cost Budgeting
- Schedule Development
- Quality Planning
- Communications Planning
- Organization Planning
- Staff Acquisition
- Risk Planning
- Procurement Planning
- Project Plan Development

Develop the project plan

- Develop project management plan
  - Concerned with creating and integrating all sub-plans into a single source of information
  - Identify the project activities.
- Scope planning
  - Concerned with how the project scope statement will be created
- Create WBS
- Scope definition
  - Concerned with actual creation of project scope statement
- Activity definition
  - Activity sequencing
  - Activity duration estimating
  - Activity resource estimating
- Determine resource requirements.

Planning processes

- Schedule development
  - Concerned with analyzing activity outputs (definition, etc.) to create project schedule
- Construct/analyze the project network.
- Cost estimating **
- Cost budgeting
  - Concerned with aggregating costs of individual activities to establish cost baseline
- Quality planning *
  - Concerned with quality standards and how to achieve them
- Human resource planning *
- Communications planning *

* indicates minimal or no coverage

** indicates optional coverage

Planning processes

- Risk management planning
  - Concerned with how to carry out risk management activities
  - Risk identification
  - Qualitative risk analysis
    - Concerned with prioritizing risks based on probability of occurrence and impact
  - Quantitative risk analysis *
  - Risk response planning
    - Concerned with mitigating risks to project objectives
- Plan purchases and acquisitions *
  - Concerned with what, when, and how of purchases and acquisitions
- Plan contracting *
- Prepare the project proposal.

Executing Process

Coordinating people and other resources to carry out the plan

- Project Plan Execution
- Scope Verification
- Quality Assurance
- Acquire project team
  - Identify and organize the project team.
  - Establish team operating rules.
  - Team Development
- Solicitation
  - Information Distribution
  - Source Selection
  - Contract Administration
- Level project resources.
- Schedule work packages.
- Document work packages.

Monitoring & Controlling Process

Monitor and control project work

- Ensuring that project objectives are met by monitoring and measuring progress and taking corrective measures when necessary
- Concerned with acquiring and assessing performance information to effect process improvements

- Integrated change control
  - Overall Change Control
  - Scope Change Control
  - Schedule Control
- Scope control – Concerned with changes to project scope
- Scope verification – Concerned with acceptance of project deliverables
- Schedule control – Concerned with changes to project schedule

Monitoring & Controlling Process

- Cost control * – Concerned with changes to the project budget
- Quality Control – Concerned with monitoring quality compliance of project results and correcting unsatisfactory results
- Manage project team – Concerned with tracking performance, providing feedback, and coordinating changes
  - Define problem-escalation process.
  - Monitor project progress versus plan.
  - Establish progress reporting systems.
  - Performance reporting * – Concerned with status, progress, and forecasting
- Install change control tools/process.
- Risk monitoring and control
- Manage stakeholders
- Contract administration *

☛ Revise project plans.

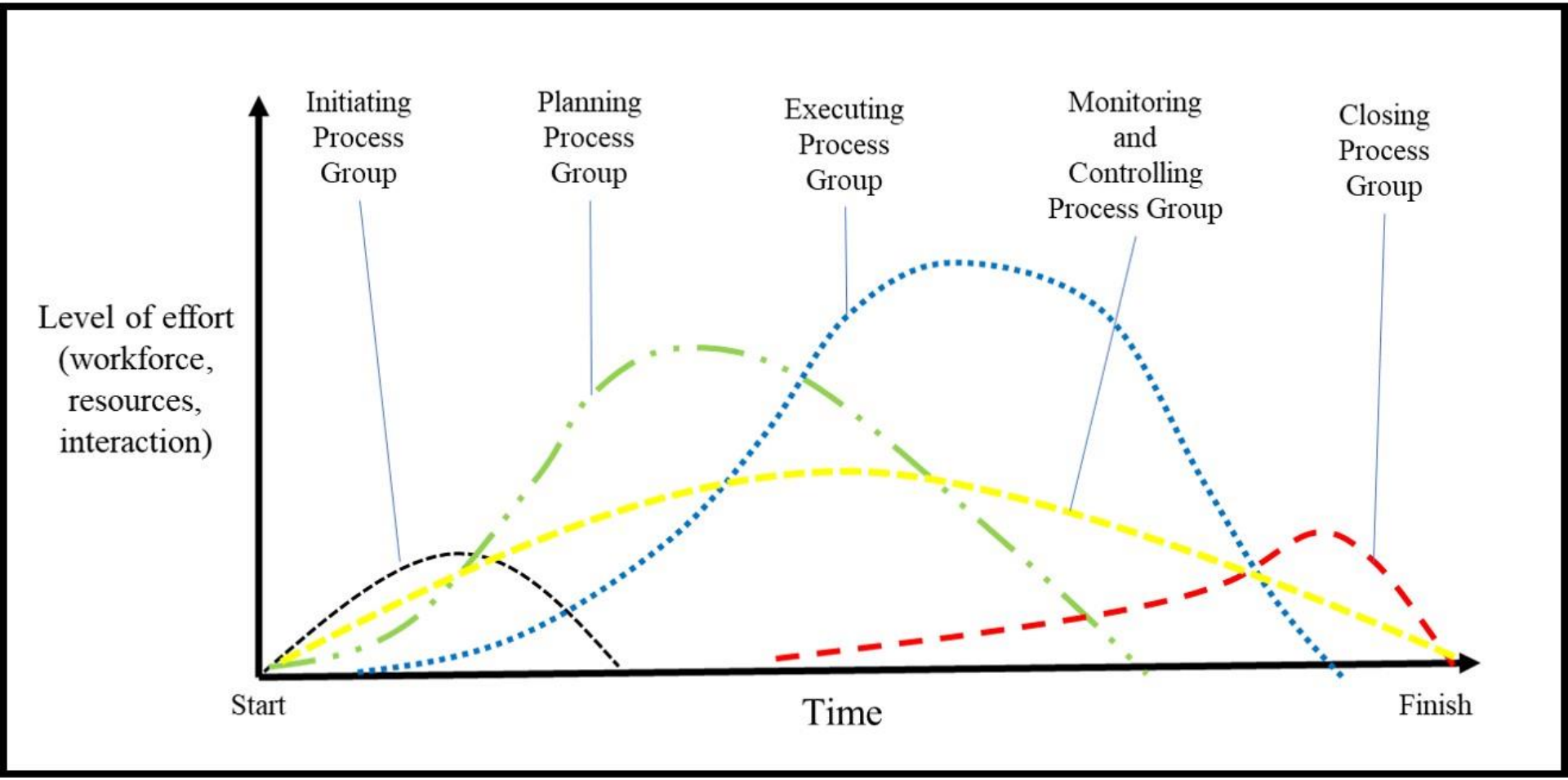
Close out the project

Formalizing acceptance of the project or phase and bringing it to an orderly end

- Administrative Closure
  - Concerned with finalizing all activities across all Process Groups
  - Complete project documentation.
  - Complete post-implementation audit.
    - Lessons learned
  - Issues final project report.
- Contract Close-out
  - Concerned with completing and settling all contracts
  - Obtain client acceptance.
  - Install project deliverables.

Phases of the Project Management

Level of Activity and Overlap of Process Groups Over Time



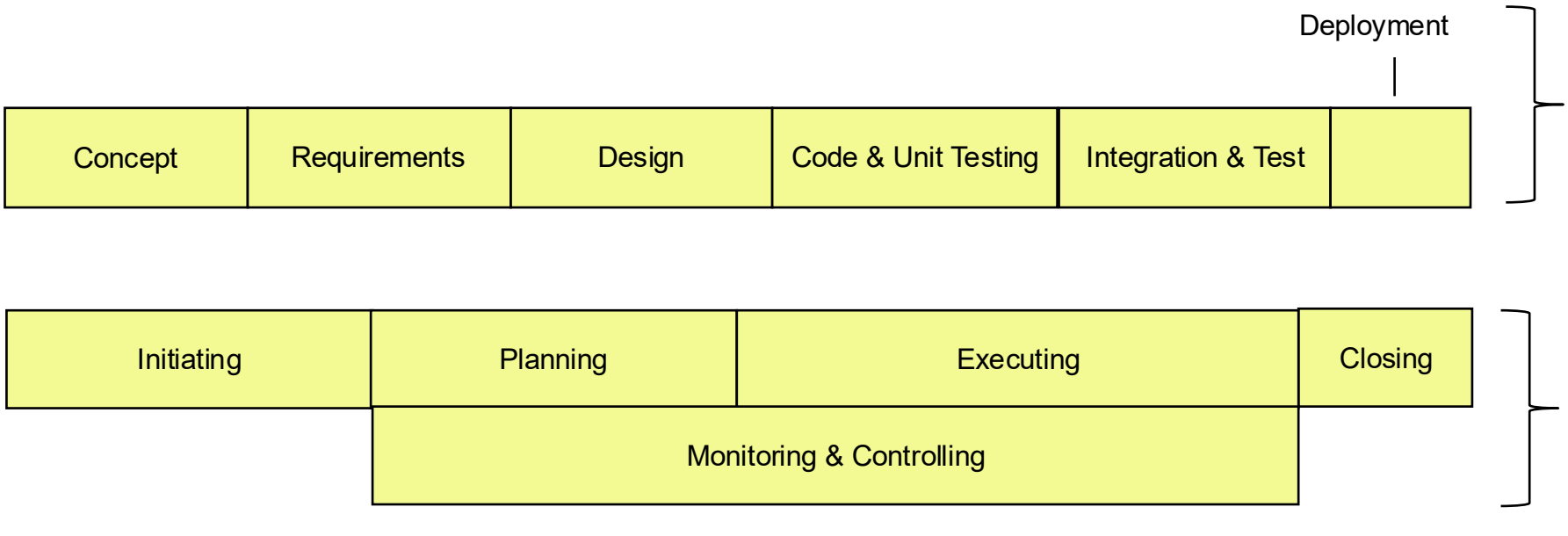
Project Processes & Their Integration

- Project Management Processes (Principles of Project Management)
  - Initiating processes (Defining)
  - Planning processes
  - Executing processes
  - Monitoring & controlling processes
  - Closing processes
- System Development Processes (Iterative/evolutionary)
  - Inception phase
  - Elaboration phase
  - Construction phase
  - Transition phase
- Integrating IT Project Processes
  - PM/IT project integration tactics

PM/IT process integration tactics

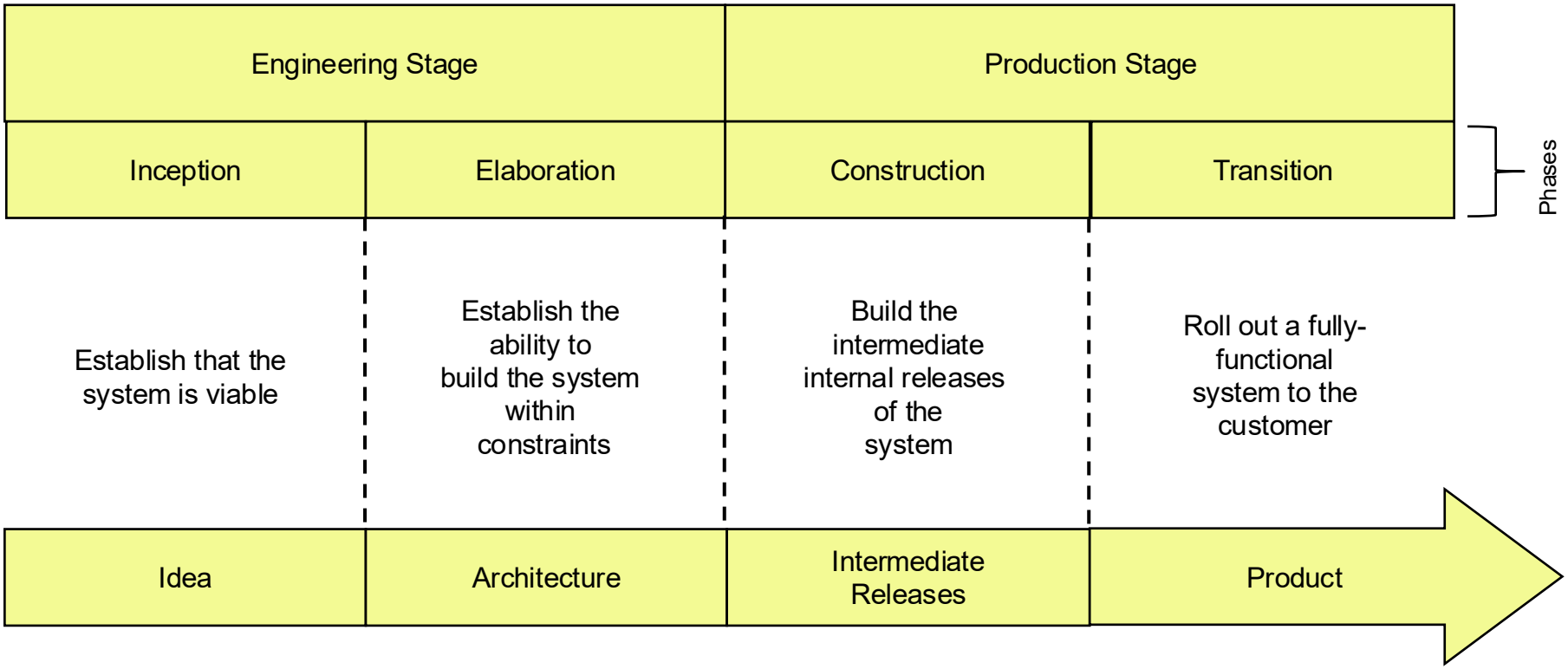
- Wherever possible, establish common policies, processes, and procedures between IT and PM groups
- Identify an integration manager to link IT and PM groups
- Use a common, integrated, consistent vocabulary that is continuously updated to facilitate inter- (as well as intra-) group communications
- Ensure that project manager possesses suitable process integration skills and is familiar with IT risks
- Involve IT analysts in development of business requirements
- Identify an ombudsman to quickly resolve issues that arise between PM and IT groups

Project & SDLC integration waterfall development model



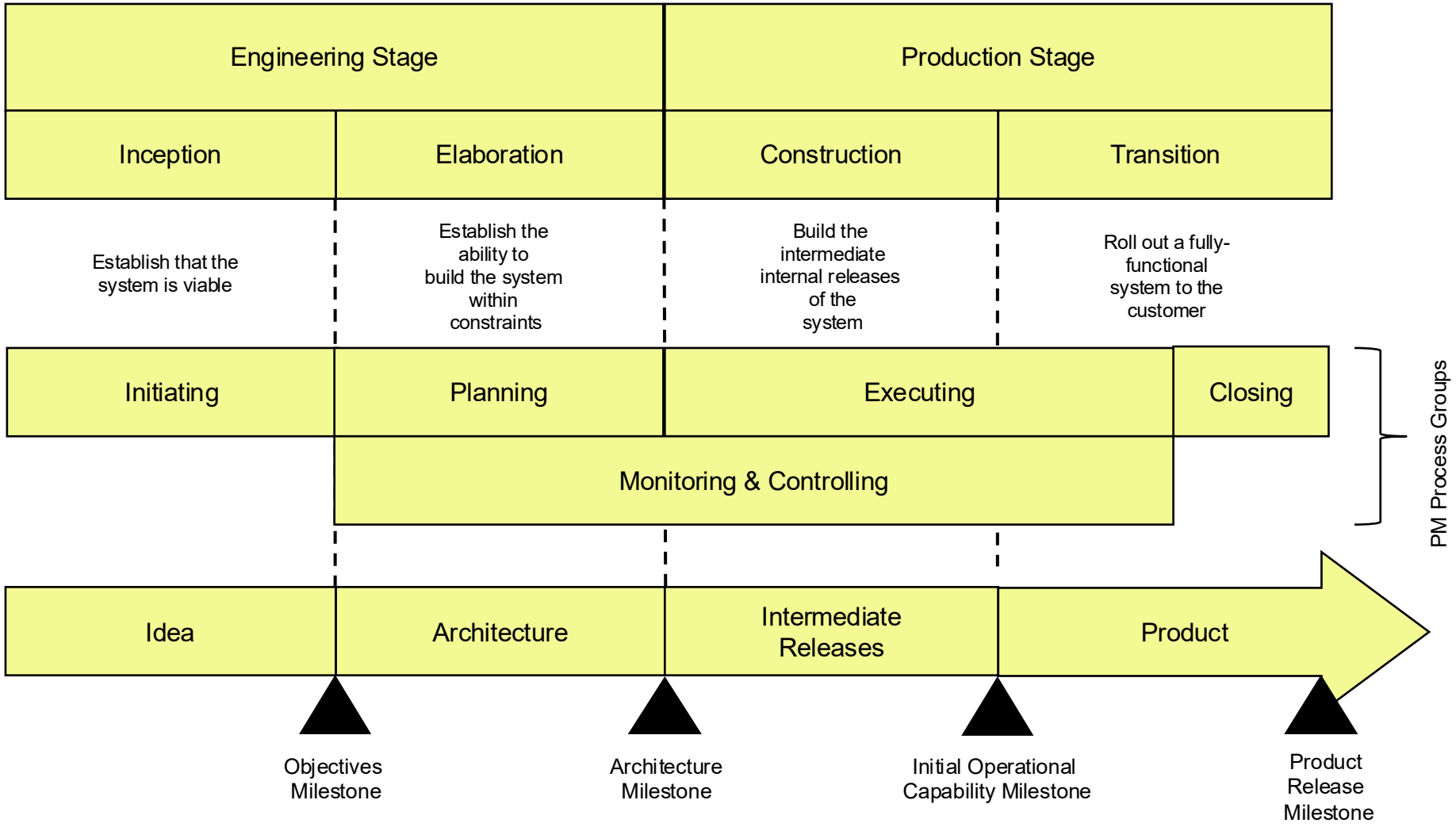
Phases in iterative* system life cycle

The stages below are repeated (iterative) – see notes



* I often interchange *iterative* & *evolutionary*

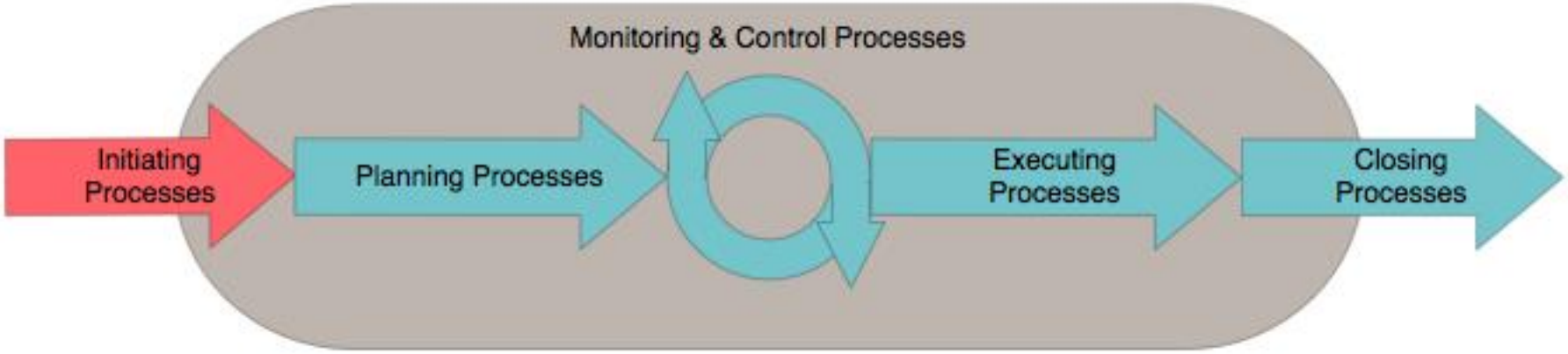
Project & SDLC integration
iterative/incremental development model



Initiating the Project

Initiating processes overview

- ***Initiating processes*** define a new project or new phase of an existing project
- Initial project scope, project stakeholders, and project manager are identified
- Key purposes are to:
  - *Align the stakeholders' expectations* with project purpose
  - *Give stakeholders visibility* into project scope and objectives
  - *Demonstrate that stakeholder participation helps* ensure project success
- All of these set the *vision* of the project: what needs to be accomplished



Adapted from Figure 2-10 *PMBOK Guide, 5th Edition*

Initiating Processes

- **Develop Project Charter**
  - This process falls under the **Project *Integration* Management knowledge area**
  - Justifies and formally authorizes a project or a project phase
  - Documents the stakeholders' initial requirements and expectations
  - Forms the basis for the partnership between the requesting (customer) and performing (supplier) organizations
- **Identify Stakeholders**
  - This process falls under the **Project *Communications* Management knowledge area**
  - Identifies all people or organizations impacted by the project
  - Documents their interests and involvement with the project, as well as their potential impact on project success
  - Forms the basis for developing a strategy to approach and involve each stakeholder to maximize positive influences and minimize negative influences

Concept Exploration

- **The "Why" phase**
- Not a "mandatory formal" phase
  - Sometimes called the "pre-project" phase
- Collecting project ideas
  - Then the "funneling" process
- **Project Justification**
  - ROI – Return on Investment

$$\text{Return on Investment (ROI)} = \frac{\text{Net Return}}{\text{Cost of Investment}}$$

  - Cost-benefit analysis
  - Competitive analysis (if appropriate)
- Initial planning and estimates

Concept Exploration

- Possibly includes Procurement Management:
  - **RFP** (request for proposal) Process
  - Vendor selection
  - Contract management
- Gathering the initial team
  - Including PM if not already on-board
- Identify the project sponsor
  - Primary contact for approval and decision making
- Potential Phase Outputs:
  - Concept Document, Product Description, Proposal, SOW, Project Charter

The Charter

Inputs, tools & techniques, and outputs

Inputs

- Project statement of work
- Business case
- Agreements
- Enterprise environmental factors
- Organizational process assets

Tools & Techniques

- Expert judgement
- Facilitation techniques

Outputs

- Project charter

Define the Project

- There is a need for clear understanding of exactly what is to be done.
- Project definition starts with the *Conditions of Satisfaction* document based on conversation with the customer.
- *Project Overview Statement* aka *Charter* or *Vision* is generated from the Conditions of Satisfaction document.
  - The *Project Overview Statement* clearly states what is to be done.
  - Once the *Project Overview Statement* is approved, the scoping phase is complete.

Note: In most cases the ***Project Overview Statement***, the ***Statement of Work***, and ***Project Charter*** are the same. Even **scope** will fit here. You can use them interchangeably.

Project Charter

- Activities
  - Define scope
  - Document Project Risks, Assumptions, and Constraints
  - Identify and Perform Stakeholder Analysis
  - Develop Project Charter
  - Obtain Project Charter Approval
- Deliverables
  - Project charter
  - Statement of work (SOW) (aka Scope)

Preliminary Scope

- Project objectives
- Product description
- Deliverables
- Constraints
- Assumptions
- Project acceptance criteria

Project Charter

- Typical outline
  - **Overview**
    - » Business need
      - Problem/opportunity
    - » Objectives
      - Project goal
    - » Method or approach
  - **General scope of work**
    - » Success criteria
  - **Rough schedule & budget**
  - **Roles & responsibilities**
  - **Assumptions, risks, obstacles**

Project charter content

- **Project purpose or justification**
  - Define the reason why the project is being done, by referring to any of the Initiating process inputs
- **Measurable project objectives** and related success criteria
  - *Scope.* Scope needed to achieve project goals and measurable criteria for scope success
  - *Time*. Goals for timely completion and specific dates for success
  - *Cost*. Goals for project expenditures and range of costs for success
  - *Quality*. Quality criteria and specific measures for criteria for success
  - *Other*. Any other objectives along with measurable criteria of success

Project charter content

- **High-level requirements**
  - Describe the high-level product capabilities that satisfy stakeholder needs and expectations. Do not include detailed requirements
    - *Example*: As a retail customer, I want to shop by either **brand** or by **product** category
    - *Example:* As the site owner, I want a retail customer to **find a stocked product** on the site within **three (3) mouse clicks**
    - *Anti-example*: As the site owner, I want the products to be displayed in **a 4- across grid** against a **light grey background**

Project charter content

- **Assumptions and constraints**
  - An ***assumption*** is "a thing that is accepted as true or as certain to happen, without proof"
    - » *Example*: The site will allow all site visitors to access all public features of the site
  - A ***constraint*** is a limitation or restriction
    - » *Example*: The site must use a hosting service for the new site

Project charter content

- **High-level project description and boundaries** [scope]
  - Provide an executive-summary-level description of the project, identify what will and will not be included in the project
    - *Example*: The site is a one-stop source for health and wellness information … It will not provide direct access to the HR site.
- **High-level risks**
  - *Risks* represent any major areas of uncertainty for the project
  - *Risks* may be internal or external
    - *Example*: Existing customers may have difficulty transitioning to new site
    - *Example*: The company does not have sufficient in-house Web design expertise to match the goals for the new site

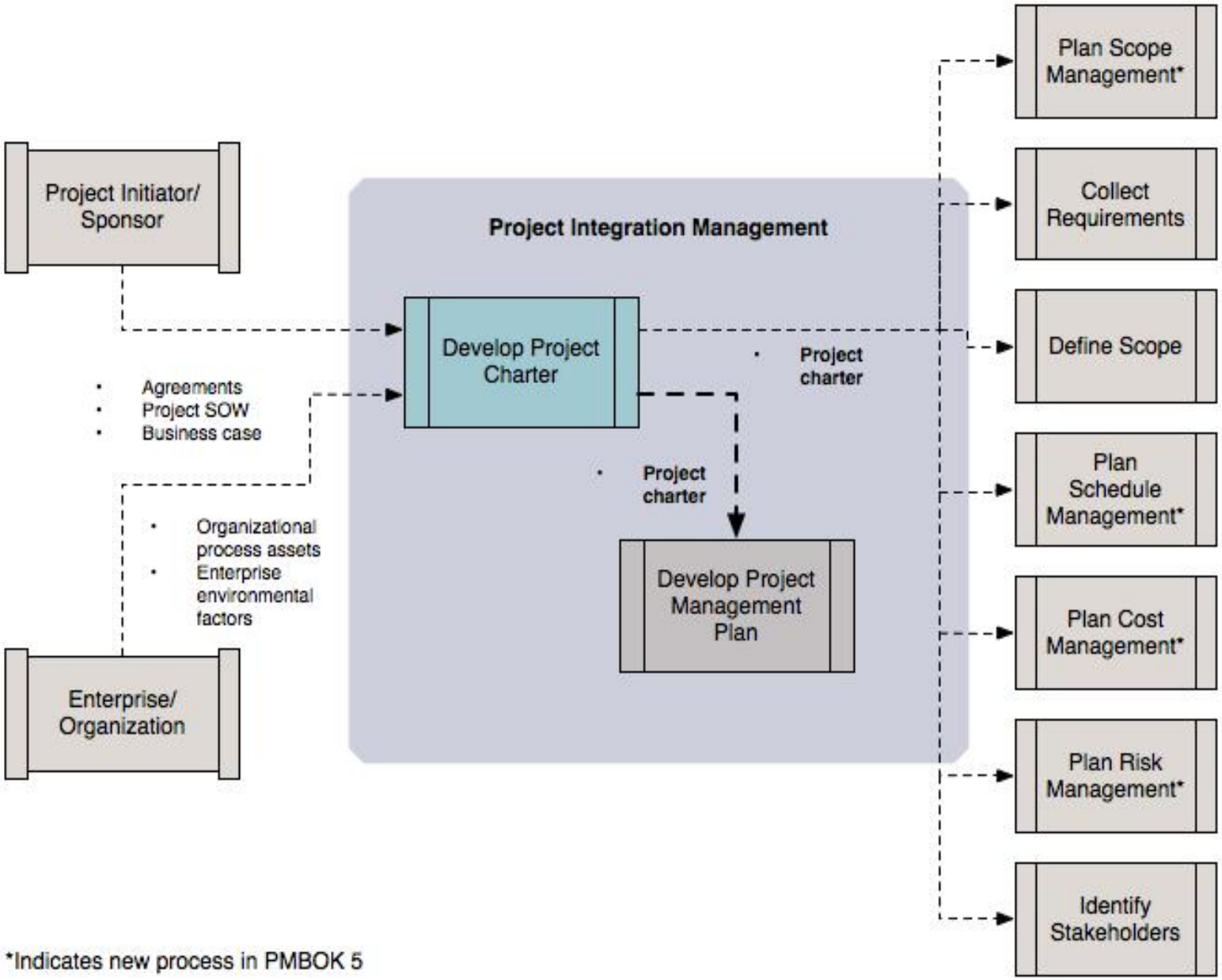
Project charter content

- Summary **milestone schedule**
  - Identify any *significant points or events* in the project, such as key deliverables, beginning or ending of phases, or product acceptance
  - Include estimated completion dates for the milestones
    - » Examples:
      - Requirements document complete: 4/30/2024;
      - Web site on-line with training: 6/30/2024
- Summary **budget**
  - Provide a rough order of magnitude (ROM) estimate of expenditures schedule for project
    - » ROM estimates may be as broad as ±100% but usually range ±35%
  - Budget should break down total expenditures by major categories (software, hardware, human resources, etc.)

Project charter content

- **Stakeholder list**
  - A *stakeholder* is "a[n] individual, group, or organization who may affect, be affected by, or perceive itself to be affected by a decision, activity, or outcome of a project"*
  - Identify a preliminary list of the most critical project stakeholders–this list will be refined later
- **Project approval requirements**
  - Identify any criteria that must be met in order for the project to be accepted by the project customer
  - *Example:* The project must implement the set of 'must-have' user stories agreed upon at project initiation

Develop Project Charter: Data Flow Diagram



Adapted from Figure 4-3 *PMBOK Guide, 5th Edition*

Charter

Tips on writing the charter

- Distribution of project by **type**
  - In-house, contract/for-hire, startup
- Distribution of project by **technology**
  - Web, Windows/Mac OS/Linux, Mobile, No platform
- Distribution by **industry**
  - Financial Services, Law, Retail
- A reminder why no two projects are same
- Most important **elements**
  - Why, who, what, what not
  - A little bit of when
- Make sure it's clear
- Some more re-purposed than others
  - Occasionally read more like business plans

Charter

- Formalities
  - Spell check.
  - Proof read
  - Make sure your name is on first page and also header and/or footer on each page.

Statement of Work (SOW)

Statement of Work (SOW)

- A description of the work required for the project; normally this is used when the project is being contracted out, but most of this is part of the Project Overview or Charter
- Sets the "boundary conditions"
- SOW vs. CSOW (Contract SOW)
  - Latter: uses legal language as part of a competitive bidding scenario
- Can be used in the final contract – be careful, be specific, be clear
- Typically done after approval (after "Go")
- Can be multiple versions
  1. List of deliverables for an RFP
  2. More detailed within final RFP
  3. Binding version from contract

SOW Template

I. **Scope of Work:** Describe the work to be done to detail. Specify the hardware and software involved and the exact nature of the work.

II. **Location of Work:** Describe where the work must be performed. Specify the location of hardware and software and where the people must perform the work

III. **Period of Performance:** Specify when the work is expected to start and end, working hours, number of hours that can be billed per week, where the work must be performed, and related schedule information. Optional "Compensation" section.

IV. **Deliverables Schedule:** List specific deliverables, describe them in detail, and specify when they are due.

V. **Applicable Standards:** Specify any company or industry-specific standards that are relevant to performing the work. Often an Assumptions section as well.

VI. **Acceptance Criteria:** Describe how the buyer organization will determine if the work is acceptable.

VII. **Special Requirements:** Specify any special requirements such as hardware or software certifications, minimum degree or experience level of personnel, travel requirements, documentation, testing, support, and so on.

S.M.A.R.T. characteristics for Goal

- Doran's S.M.A.R.T. characteristics provide the criteria for a goal statement:
  - **S**pecific: Be specific in targeting and objective.
  - **M**easurable: Establish measurable indicator(s) of progress.
  - **A**ssignable: Make the object assignable to one person for completion.
  - **R**ealistic: State what can realistically be done with available resources.
  - **T**ime-related: State when the objective can be achieved; that is, duration

The Project Definition Statement : PDS

The Project Definition Statement : PDS

- Just as the customer and the project manager benefit from the Charter, the project manager and project team can benefit from a closely related document, which we call the *Project Definition Statement (PDS)*.
- The PDS uses the same form as the Charter but incorporates *considerably more detail*. The detailed information provided in the PDS is for the use of the project manager and the project team.

Product overview document content

- *Product Name*
- *Product Vision Statement.* Include both product problem statement and product position statement
- *Dedicated Team.* List names of team members
- *Project Manager*
- *Customer/Product Owner.* Customer or customer representative
- *Architecture.* Specify if constrained; else, to be determined by team
- *Features Backlog.* High-level list of major features
- *Product Roadmap.* Releases with themes and corresponding features
- *Risks/Opportunities.* Consider market, project, and product aspects
- *Success Criteria.* What the customer considers most critical criteria
- *Flexibility Matrix.* Trade-off matrix of time, resources, and objectives

Stakeholders

Project stakeholders

- Project stakeholders are **individuals** or **organizations** who have influence over, or are influenced by project execution or completion
- Different stakeholders have varying amounts of influence, responsibility, or authority over a project
- Stakeholders can have a positive, neutral, or negative influence on a project
- Identifying all stakeholders associated with a project may be difficult
  - Stakeholders that are overlooked almost inevitably have a negative impact on project

Interactions / Stakeholders

As a PM, who do you interact with? **Project Stakeholders**

- External people
  - Project sponsor
  - Executives
  - Customers
  - Contractors
  - Functional managers
  - Users
- Team
  - Architects
  - System Engineers
  - Designers
  - Developers
  - Testers
  - Documenters

Key project stakeholder roles

- *Customer.* Person or organization that acquires product
- *User*. Person or organization that uses product
- *Performing organization*. Organization performing work of project
- *Project manager*. Responsible for managing project
- *Project management team*. Individuals directly involved in project management activities
- *Project team members.* Individuals performing work of project
- *Sponsor*. Entity providing financial resources for project
- *Influencers.* Entities indirectly affecting project
- *PMO*. Project management office. Responsible for centralized and coordinated management of all projects under its supervision

Significant project stakeholders

- *Functional managers*. Manage within functional or administrative areas of the business, such as human resources accounting, or procurement. Do not deal directly with products or services
- *Operations management.* Manage within core business areas, such as research and development, design, manufacturing, testing, or maintenance. Deal directly with producing and maintaining products
- *Sellers.* External companies or individuals that enter into contractual agreements to provide components or services necessary for project. Also known as vendors, suppliers, or contractors
- *Business partners.* External companies or individuals that have a closer relationship with enterprise, providing expertise or filling specific roles such as installation, training, or support

Software System Stakeholders

Each stakeholder has different concerns:

Customer	Manager	Architect	Developer	Tester
Requirements Cost Schedule Performance Reliability Security	Cost Schedule Requirements Process Resources	Maintainability Portability Feasibility Reusability Extensibility Flexibility The ***ilities***	Components Connectors Class/Pattern Data flow Reuse Flexibility Extensibility	Functionality Requirements Regression Tools Simulators

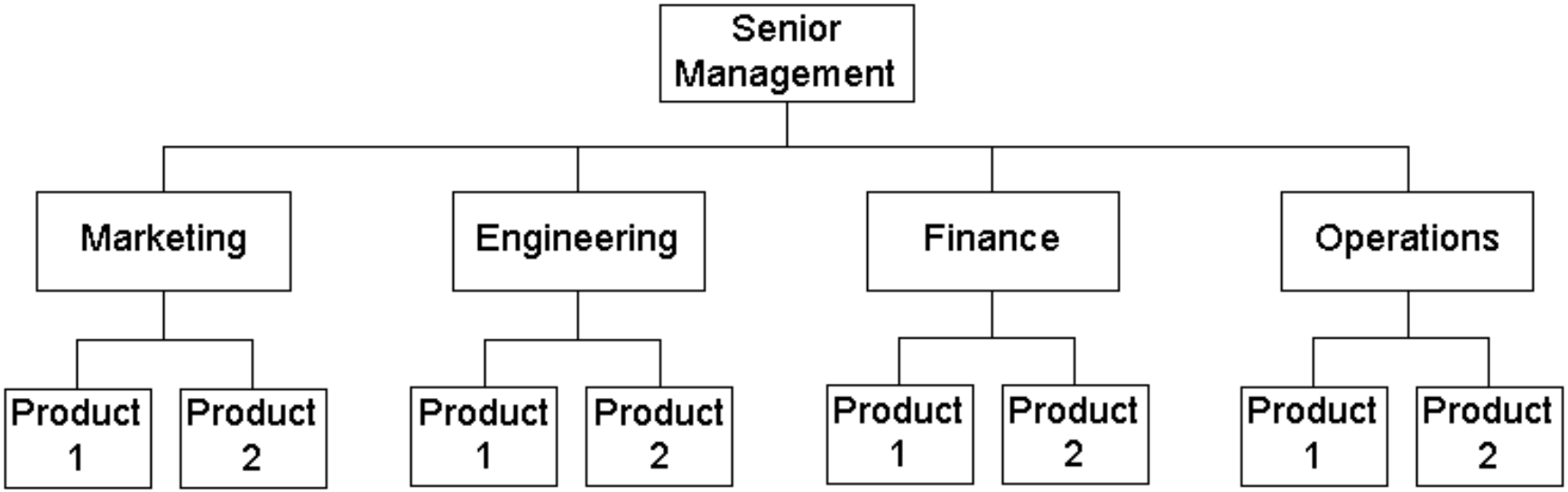
Stakeholder Triad

1. **Function Representative**
   - The ‘business person’, Business Analyst (BA)
   - Or SME: Subject Matter Expert
2. **Executive Sponsor**
   - Project's visionary & champion
   - Also the ‘General’, ‘Fall Guy’, and ‘Minesweeper’
   - Not the PM, ’Santa Claus’, or the ‘Tech Guy’
3. **Project Manager**
   - The ‘Linchpin’
   - Must be ‘multi-lingual’

Organizational Structures

- **Functional**
  - Engineering, Marketing, Design, etc.
  - P&L from production
- **Projectized**
  - Project A, Project B
  - Income from projects
  - PM has P&L responsibility
- **Matrix**
  - Functional and Project based
  - Program Mgmt. Model
  - Shorter cycles, need for rapid development process

Functional Organization



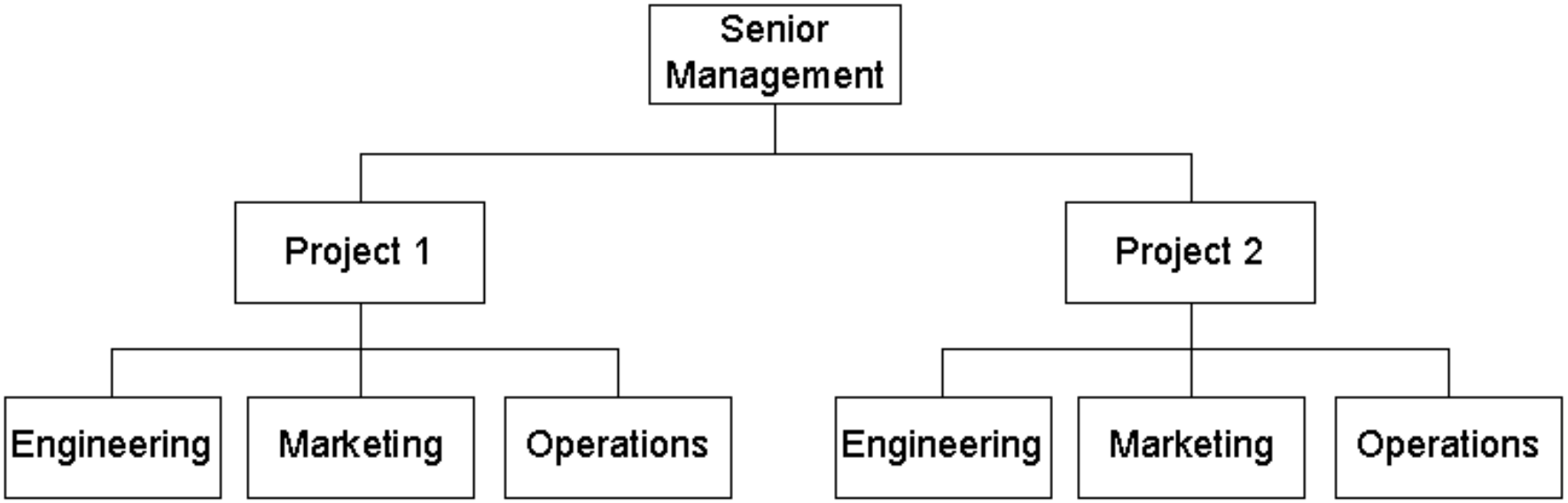
Pros

- Clear definition of authority
- Eliminates duplication
- Encourages specialization
- Clear career paths

Cons

- "Walls": can lack customer orientation
- "Silos" create longer decisions cycles
- Conflicts across functional areas
- Project leaders have little power

Projectized Organization



Pros

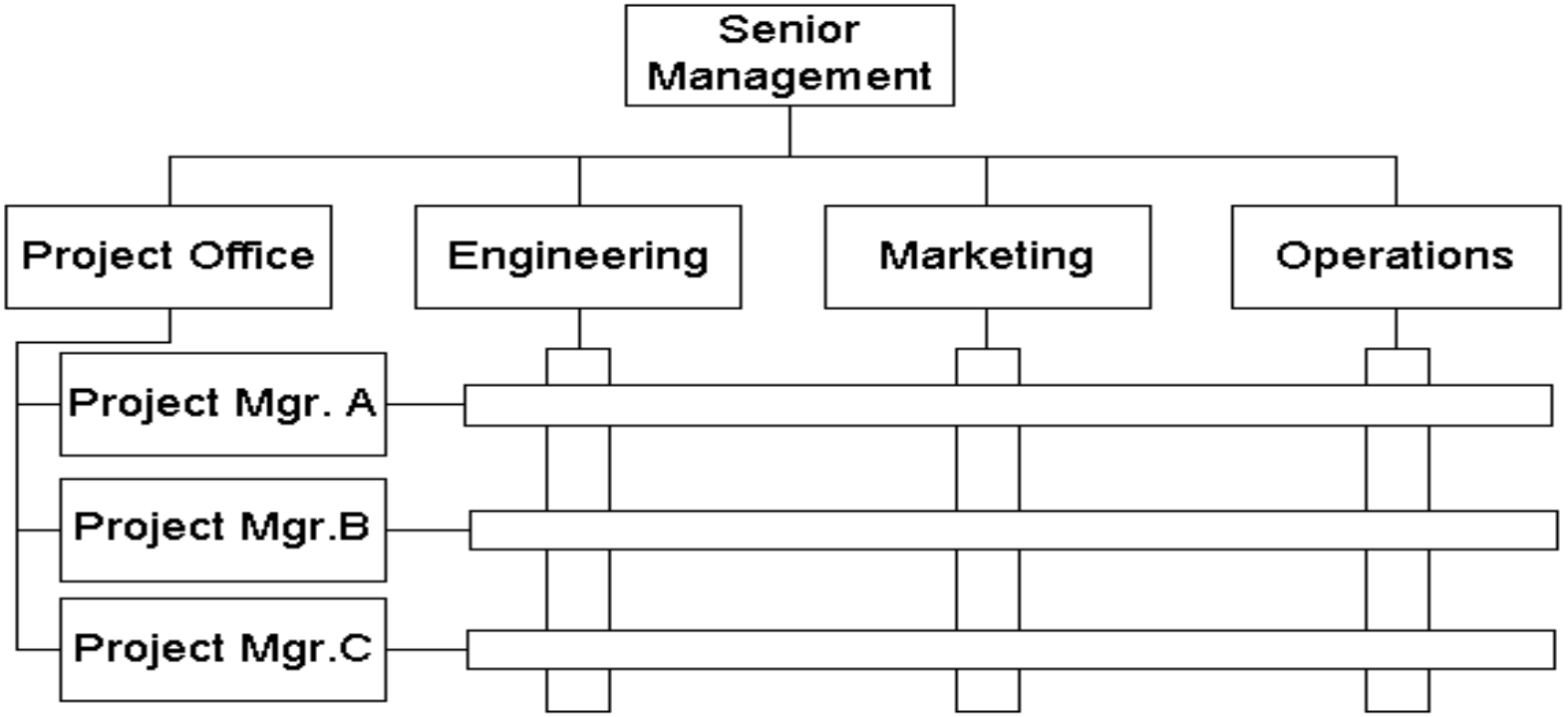
– Unity of command
– Effective intra-project communication

Cons

– Duplication of facilities
– Career path

Examples: defense avionics, construction

Matrix Organization



Pros

– Project integration across functional lines
–Efficient use of resources
–Retains functional teams

Cons

– Two bosses for personnel
– Complexity
– Resource & priority conflicts

Matrix Forms

- Weak, Strong, Balanced
- Degree of relative power
  - Weak: functional-centric
  - Strong: project-centric

Organizational Structure – Influences on Projects

Organization Type **Project Characteristics**	Functional	Matrix			Projectized
		Weak Matrix	Balanced Matrix	Strong Matrix	
Project Manager's Authority	Little or None	Limited	Low to Moderate	Moderate To High	High to Almost Total
Percent of Performing Organization's Personnel Assigned Full-time to Project Work	Virtually None	0-25%	15-60%	50-95%	85-100%
Project Manager's Role	Part-time	Part-time	Full-time	Full-time	Full-time
Common Title for Project Manager's Role	Project Coordinator/ Project Leader	Project Coordinator/ Project Leader	Project Manager/ Project Officer	Project Manager/ Program Manager	Project Manager/ Program Manager
Project Management Administrative Staff	Part-time	Part-time	Part-time	Full-time	Full-time

PMBOK Guide, 2000, p. 19

Project Planning

Software Project Planning

The overall goal of project planning is to establish a pragmatic strategy for controlling, tracking, and monitoring a complex technical project.

OR

A Plan is the strategy for the successful completion of the project. It's a description of the project steps that produce increasing maturity of the products or processes produced by the project.

Why?

So the end result gets done on time, with quality!

Reasons people don't plan

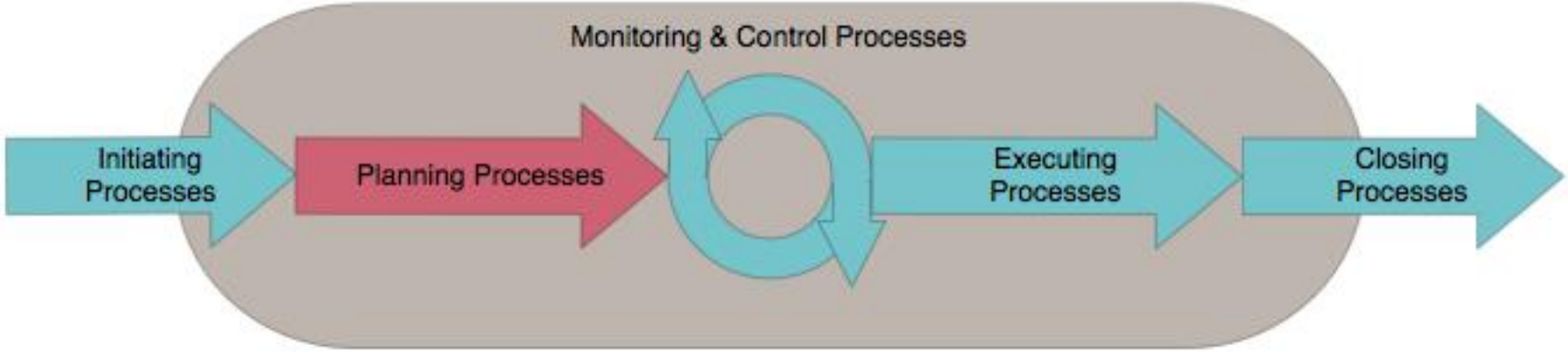
- Don't believe in planning
  - Management or organization culture does not support planning
- Find the process painful
  - If you do plan, the pain will be greatest early and will diminish with time
  - If you don't plan, you defer the pain (and it will usually be greater…)
- Don't have time to plan!
  - Consider the simple task of getting yourself to class
  - Now consider having the responsibility of getting all the other people to class on time

Planning is essential to control

- An effective way to exert control is to:
  - Know where you are
  - Know where you are supposed to be
  - Take corrective action if there is a difference between the two
- Note:
  - You have to have a plan to know where you are supposed to be
  - If you have no plan, you have no control
    - *Example*: Commercial airliner flying from Chicago to Tokyo
- Planning and control
  - Two phases both needed

Planning processes

- **Planning processes** determine the total project scope, define or refine the project objectives, and develop the course of action to achieve the objectives
- Planning employs *progressive elaboration*: the process of revisiting planning (and possibly initiating) processes as additional project information becomes available
- The planning processes encompass project integration management, project scope management, project time management, project risk management, and project stakeholder management



Planning

- **Scoping**
  - What is the problem?
- **Estimation**
  - How much will it cost?
- **Schedule**
  - How long will it take?
- **Resources**
  - How many people will it take?
- **Risk**
  - What might go wrong?
- **Control Strategy**

Planning processes and knowledge areas

Knowledge Area Processes	Planning Process Group
Project Integration Management	• Develop Project Management Plan
Project Scope Management	• Plan Scope Management • Collect Requirements • Define Scope • Create WBS
Project Time Management	• Define Activities • Sequence Activities • Estimate Activity Resources • Estimate Activity Durations • Develop Schedule
Project Risk Management	• Identify Risks • Perform Qualitative Risk Analysis
Project Stakeholder Management	• Plan Stakeholder Management

Primary Planning Steps

1. Identify project scope and objectives
2. Define and Record Requirements
3. Identify project organizational environment
   - Analyze project characteristics
   - Identify Project Team and Define Roles and Responsibilities
4. Identify project products and activities
   - Create the WBS
   - Estimate effort for each activity
   - Allocate resources
   - Schedule deliveries and milestones

Primary Planning Steps

5. Develop Change Management Plan
6. Identify Risks and Define Risk Strategies
7. Review and communicate plan
8. Obtain Plan Approval
9. Conduct Kick-off Meeting

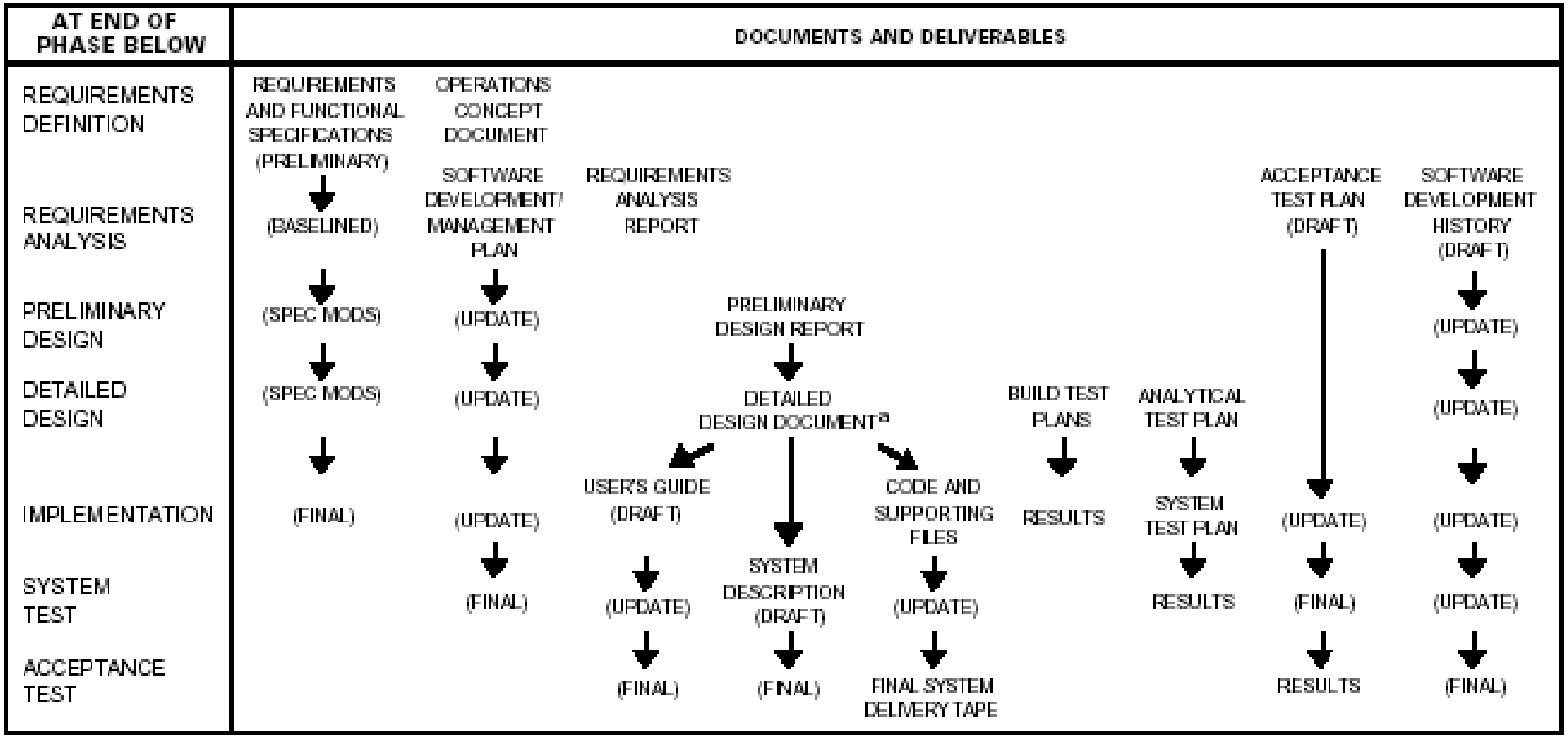
Project Planning Task Set – I

- Establish project scope
- Determine feasibility
- Define required resources
  - Determine required human resources
  - Define reusable software resources
  - Identify environmental resources
- Estimate cost and effort
  - Decompose the problem
  - Develop two or more estimates using size, function points, process tasks or use-cases
  - Reconcile the estimates

Project Planning Task Set – II

- Develop a project schedule
  - Scheduling is considered in detail later.
    - » Establish a meaningful task set
    - » Define a task network
    - » Use scheduling tools to develop a timeline chart
    - » Define schedule tracking mechanisms
- Analyze risks
  - Risk analysis

Plans Evolve Over Time



NASA's “Manager's Handbook for Software Development”

Software Development Plan (SDP)

Software Project Management Plan (SPMP)

- Some consider it the most important document in the project (along with requirements document)
  - Can be seen as an aggregation of other core documents
- Evolves over time as pieces come together

SDP / SPMP

Fundamental Sections

- Project overview
- Deliverables
- Project organization
- Managerial processes
  - Communication management plan
  - Milestones
- Technical processes
- Budget
- Schedule

Deliverables

- List of items to be delivered
  - Must be tangible items
- Sample deliverables
  - The product – the actual software, in the installable format
  - Product documentation
  - Reports and planning documentation
- Most projects are driven by deliverables, so you need several
- **Project** deliverables (aka documents) are included

Communications Management Plan

- Often a section of Software Project Management Plan (SPMP)
- Describes information flow to all parties
  - Gathering and distributing information
- Status meetings
  - Monthly, Weekly, Daily?
  - Status reports are vital
- Stakeholder Management Plan

Planning Documents

- Project ROI Analysis
  - Business Case (include competitive analysis if appropriate)
- Project Charter
  - Statement of Work (SOW)
- Software Project Management Plan (SPMP)
  - Software Development Plan (SDP)
    - Schedule
  - Communications Management Plan
- Budget
- Responsibility Assignment Matrix (RAM)
- Risk Management Plan

Example of Responsibility Assignment Matrix (RAM)

Following is an example of a responsibility assignment matrix for a new product launch project.

Task	Responsible	Accountable	Consulted	Informed
Conduct market research	Marketing team	Product manager	Sales team	Executive team
Design product packaging	Design team	Product manager	Marketing team	Executive team
Source materials for production	Procurement team	Product manager	Design team	Executive team
Set up production line	Operations team	Product manager	Procurement team	Executive team
Create marketing campaign	Marketing team	Product manager	Sales team	Executive team

Planning Documents

Other documents you may want/need

- Software Quality Assurance Plan (SQAP)
  - Software Process Improvement Plan
- Software Configuration Management Plan (SCMP)
- Migration Plan
- Operations Plan

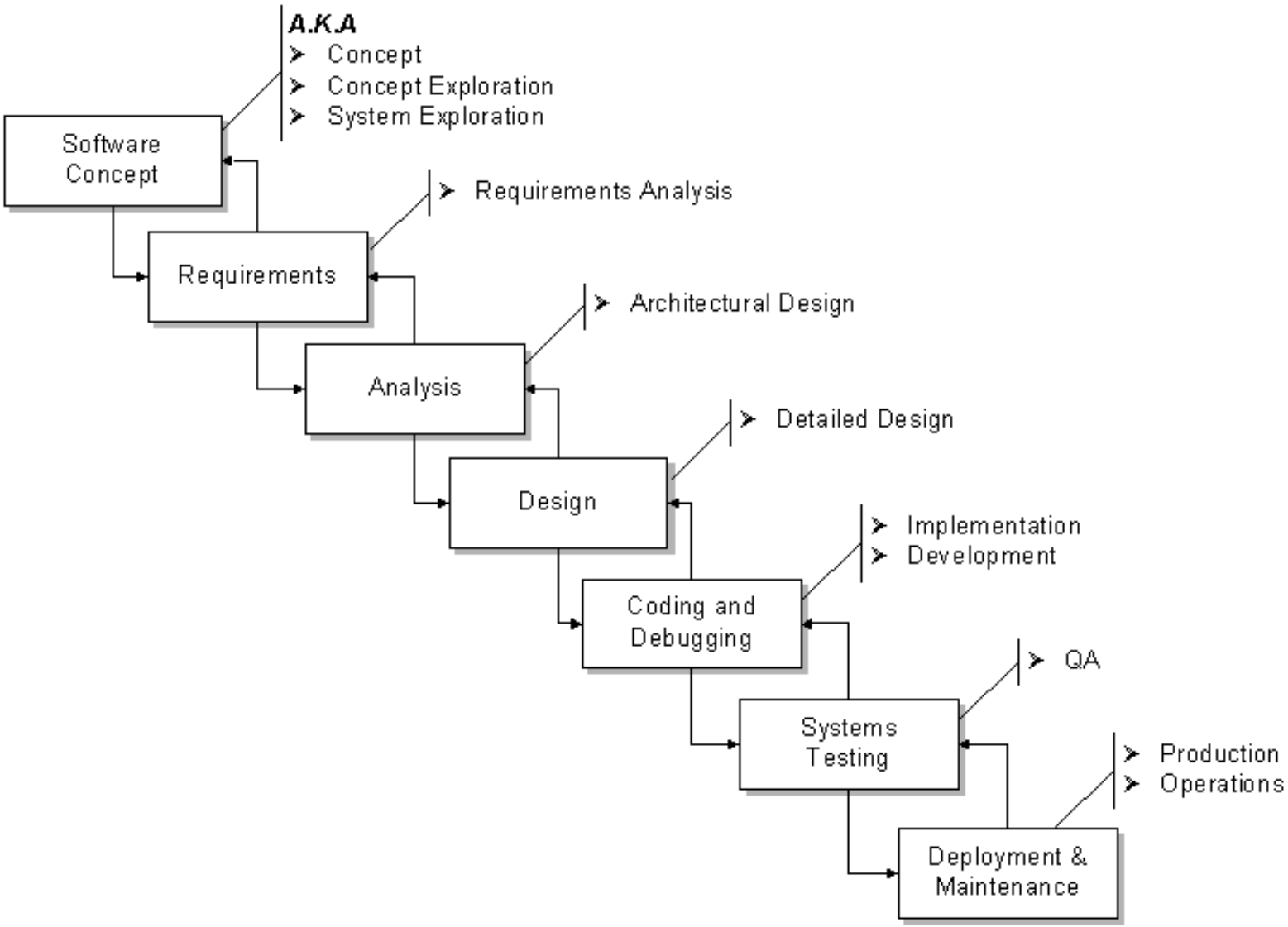
Planning Documents

- You (the PM) need to choose which documents are appropriate
- Docs do not have to be lengthy
- Small Set:
  - Software Development Plan
  - Risk Management Plan
  - Software Quality Assurance Plan
  - Software Configuration Management Plan

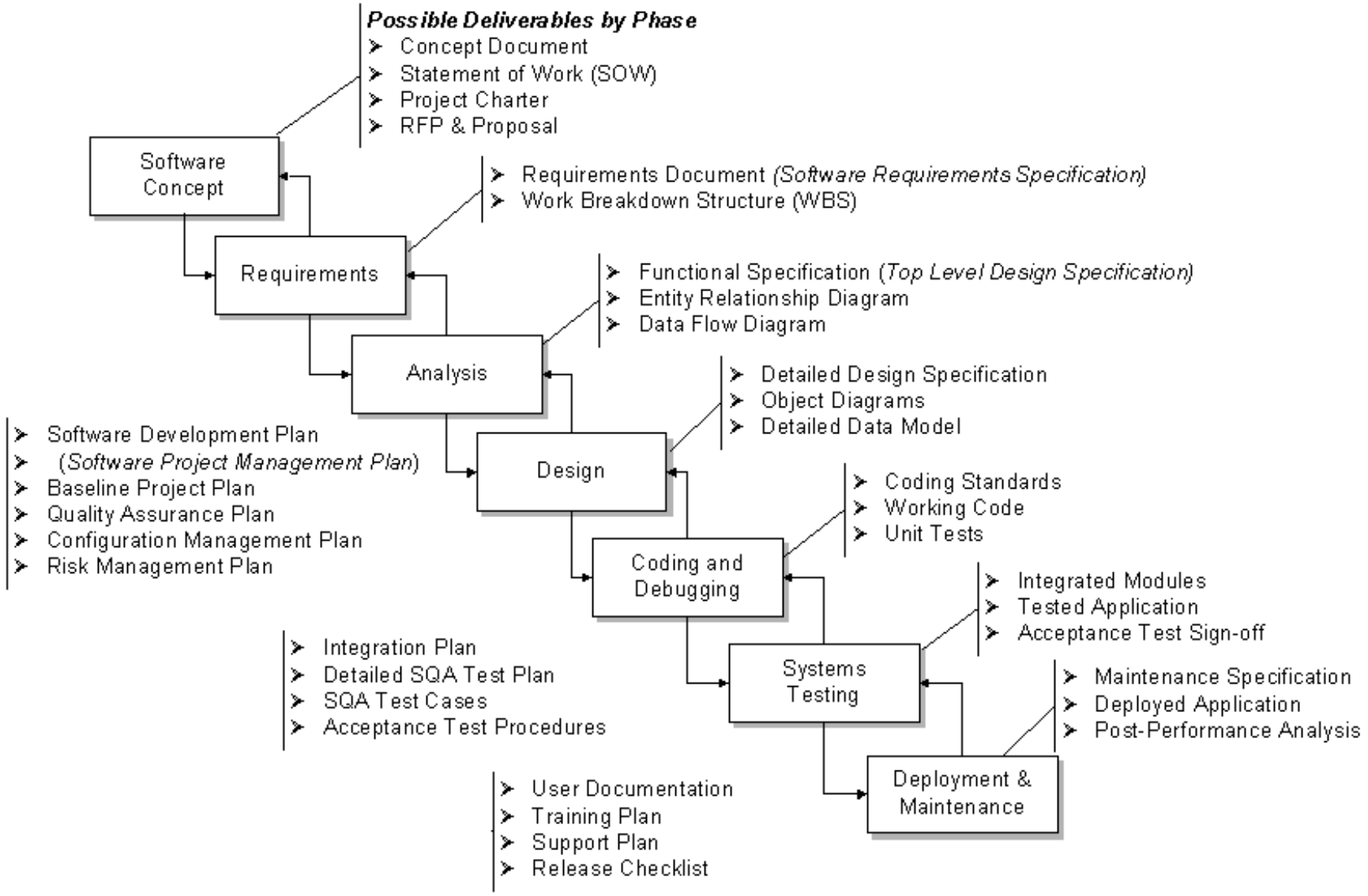
Product Documents

- Statement of Need
- System Interface Specification
- Software Requirements Specification
- Software Design Specification
- Software Validation & Verification Plan
- User Documentation
- Support Plan
- Maintenance Documentation

Project Phases



Potential Deliverables by Phase



Time Allocation by Phase

- Remember the 40-20-40 Rule
  - Specification-Implementation-Test

	Planning	Code & Unit Test	Integration & Test
Commercial DP	25%	40%	35%
Internet Systems	55%	15%	30%
Real-time Systems	35%	25%	40%
Defense Systems	40%	20%	40%

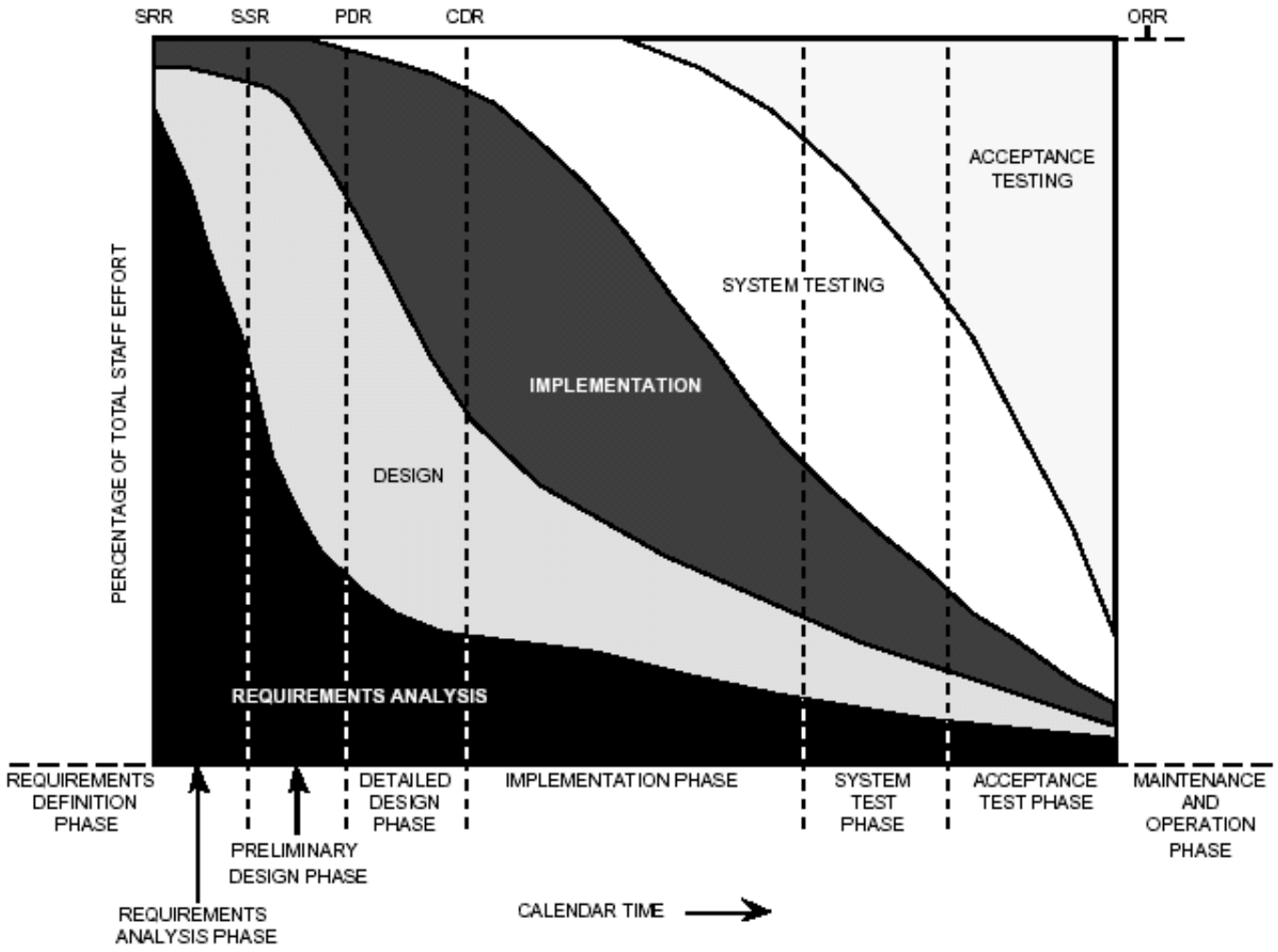
Bennatan, E.M, "On Time Within Budget"

Time Allocation by Phase

Activity	Small Project (2.5K LOC)	Large Project (500K LOC)
Analysis	10%	30%
Design	20%	20%
Code	25%	10%
Unit Test	20%	5%
Integration	15%	20%
System test	10%	15%

McConnell, Steve, “Rapid Development”

Activities by % of Total Effort



NASA's “Manager's Handbook for Software Development”

Conventional planning approach

- Planning seems to be a straightforward process:
  - Determine the tasks to be done
  - Determine the order of the tasks
  - Execute the tasks in the proper order
- Uncertainty can, and usually does, disrupt this flow
  - Not every task can be identified before the project starts
  - Contingent factors—internal or external—add, delete, or modify tasks
  - Task ordering is changed due to new or unforeseen dependencies
- At their very foundations, conventional planning approaches implicitly assume an unrealistic model of human cognition and behavior, while ignoring or underestimating real-world risk

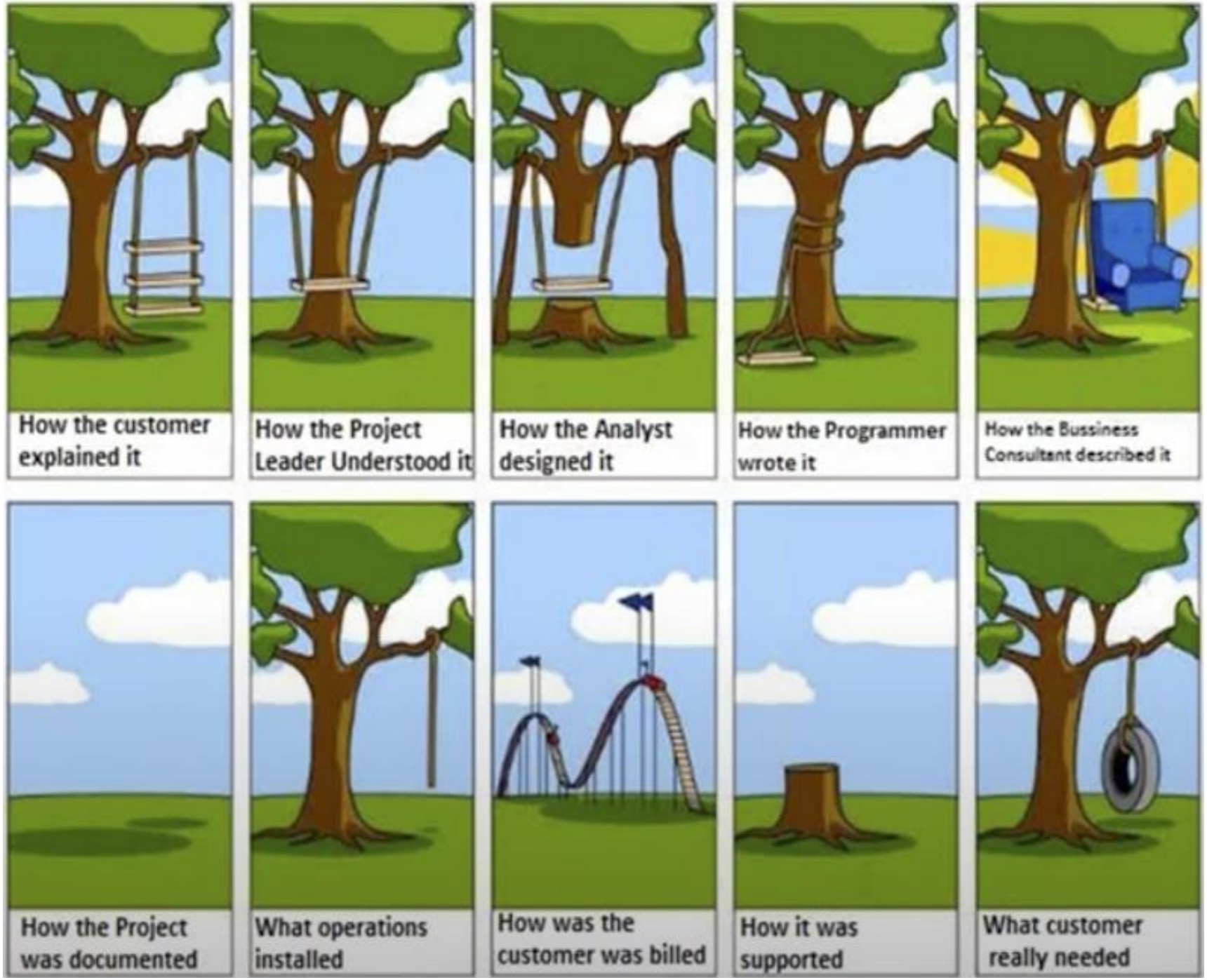
Planning in an IID methodology

- In contrast to big, up-front planning or even planning in conventional iterative and incremental methodologies, an IID methodology spreads planning out over the entire project lifecycle
- Planning in an IID methodology:
  - Distributes planning from the top-level stakeholders to the individual developers—each plans at the appropriate scale
  - Delays fine-grained planning to 'just-in-time' before corresponding task or activity
  - Is low-overhead— 'barely sufficient' planning documents are lightweight and the majority are produced 'as-needed' rather than as a matter of protocol
  - Is resilient to requirements changes—it embraces change rather than attempts to prevent or avoid it

Project scope management

Scope

- Project scope management is one of the most critical project management knowledge (*skill*) areas
- Scope defines
  - *All* the work that is required to complete the project successfully and
  - *Only* the work that is required, no more, no less
- This latter point is critical: project scope management defines and controls what ***is*** and ***is not*** part of the project work


How the customer explained it
How the Project Leader Understood it
How the Analyst designed it
How the Programmer wrote it
How the Bussiness Consultant described it
How the Project was documented
What operations installed
How was the customer was billed
How it was supported
What customer really needed

Scope planning

- Scope planning is concerned with choosing the most appropriate, most effective approach to managing the scope of the project
- Scope planning defines how to:
  1. Define the scope
  2. Develop the detailed project scope statement
  3. Define and develop the WBS
  4. Verify project scope
  5. Control project scope
- Scope planning takes two primary inputs: the project charter and the preliminary project scope statement
- Scope planning results in a *project scope management plan*

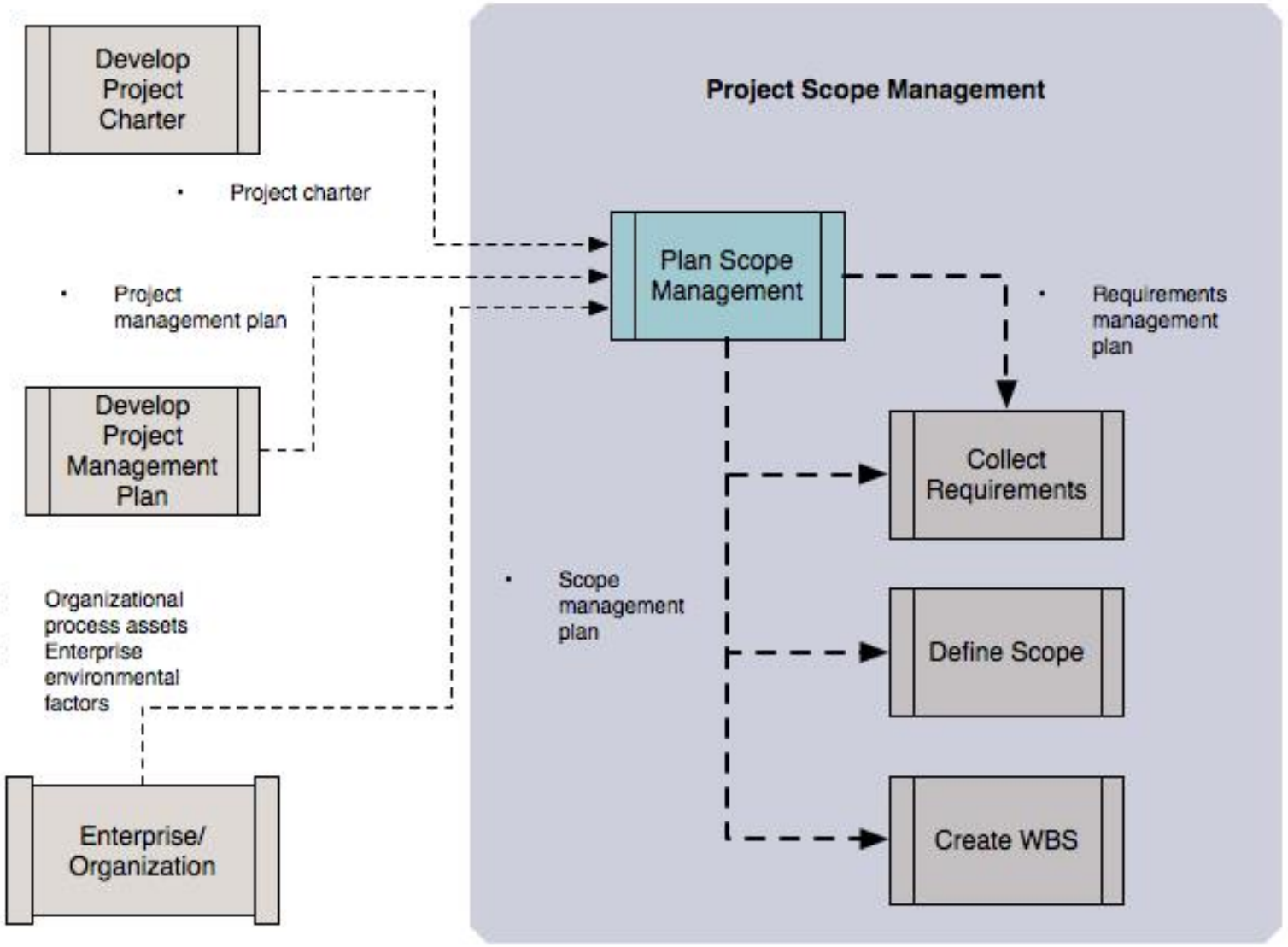
Scope management plan

- According to PMBOK 5: "The scope management plan can be formal **or** informal, broadly framed **or** highly detailed, based on the needs of the project."
  - This course adopts the informal, broadly-framed perspective
- The components of a scope management plan include:
  1. Process for preparing a detailed project scope statement
  2. Process that enables the creation of the WBS from the detailed project scope statement
  3. Process that establishes how the WBS will be maintained and approved
  4. Process that specifies how formal acceptance of the completed project deliverables will be obtained
  5. Process to control how requests for changes to the detailed project scope statement will be processed—this process is directly linked to the Perform Integrated Change Control process

Requirements management plan

- Components of the requirements management plan can include, but are not limited to:
  - How requirements activities will be planned, tracked, and reported
  - Configuration management activities such as:
    - How changes to the product will be initiated
    - How impacts will be analyzed, how they will be traced, tracked, and reported
    - Authorization levels required to approve these changes
  - Requirements prioritization process
  - Product metrics that will be used and the rationale for using them
  - Traceability structure to reflect which requirement attributes will be captured on the traceability matrix

Plan Scope Management: Data Flow Diagram



Process: Collect Requirements

- Recall the most critical lesson from the Standish Group's 2009 CHAOS report:
  - **Requirements, requirements, requirements**
    - » Requirements are unclear, incomplete, or the project management methodology does not accommodate changing requirements effectively
- Collecting initial requirements is a critical first step in managing requirements over the course of the project
  - In an iterative/incremental or adaptive/agile project, requirements collection continues throughout the life cycle of the project

Types of requirements

- ***Business requirements***. Describe the high-level needs of the organization as a whole
  - Examples: *Increase e-commerce purchases by 25%*
- ***Stakeholder requirements***. Describe the needs of a particular stakeholder, especially with respect to external stakeholders
  - Example: *As the site owner, I want the site supply chain applications to interface with the Web site*
- ***Functional requirements***.* Describe the behavior of the product
  - Example: *As a retail customer, I want to be able to shop by brand or product category*
- ***Nonfunctional requirements***.* Describe the behavioral qualities required for the product to be effective
  - Example: *As the Web site owner, I want the site to be available 99.99% of the time over a 1-year period*

*PMBOK 5 groups functional and nonfunctional requirements under the heading 'Solution Requirements'

Types of requirements

- ***Transition requirements***. Describe temporary capabilities, such as data conversion and training requirements, needed to transition from the current to the future state
  - *Example*: *As a data entry clerk, I want to be able to use the keyboard shortcuts from the previous order system, so that I can maintain my level of productivity*
- ***Project requirements***. Describe the actions, processes, or other conditions the project needs to meet
  - *Example*: *Project must use the hybrid plan-driven, iterative and agile methodology*
- ***Quality requirements***. Capture condition or criteria needed to validate the successful completion of a project deliverable
  - *Example*: *As the site owner, I want a retail customer test group to be able to successfully search for and find a requested product within 30 seconds*

Process: Create WBS

- **WBS –** Work Breakdown Structure. Technique for describing all work in a project.
- **PERT** – Program Evaluation and Review Technique. A well-entrenched technique for scheduling.
- **CPM** – Critical Path Method. Used with PERT to determine problems in scheduling.
- **Gantt Charts** – bar chart that graphically displays project schedule

The Work Breakdown Structure

- The **Work Breakdown Structure (WBS)** is a hierarchical description of the work that must be done to complete the project as defined in the Project Overview Statement (POS).
- The WBS terms
  - ***Activity***: An activity is simply a chunk of work.
  - ***Task*****:** A task is a smaller chunk of work.
  - ***Milestone***: a task of zero duration. Usually an external event. Can be used to mark completion of an activity or phase.

Decomposition

- Decomposition is usually performed in a top-down, hierarchical manner as expressed by the Work Breakdown Structure (WBS)
- Creating the *Work Breakdown Structure* (WBS) is the process of decomposing project deliverables and work into smaller, more manageable components

Work Breakdown Structure

- A Work Breakdown Structure (WBS) captures all the work of a project in an organized way.
  - Portrayed graphically as a hierarchical tree,
  - A tabular list of "element" categories and tasks or the indented task list that appears in your Gantt chart schedule.
- Breaking Large, complex projects into progressively smaller pieces
  - A collection of defined "work packages" that may include a number of tasks.
  - Continue to refine work until packages are of a suitable size
- A work package can include design, coding, testing, etc.

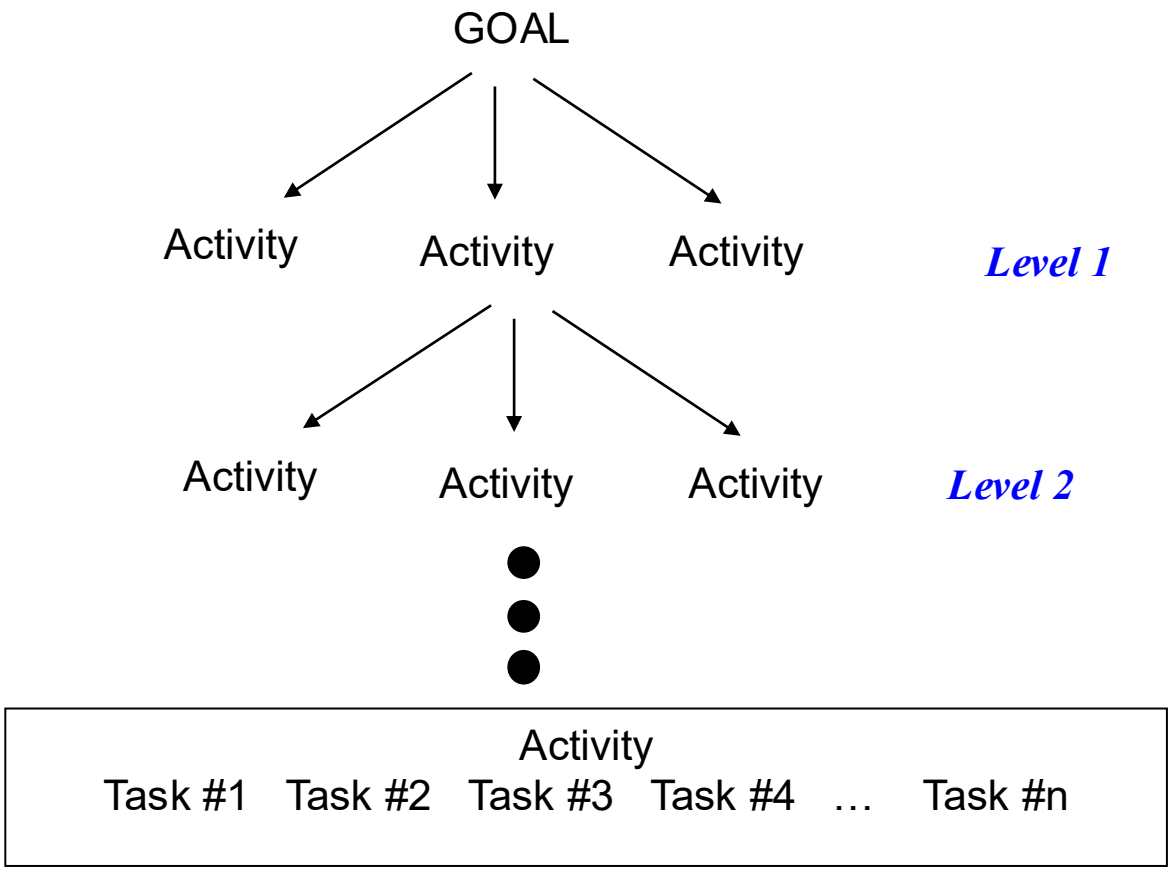
Decomposition activities

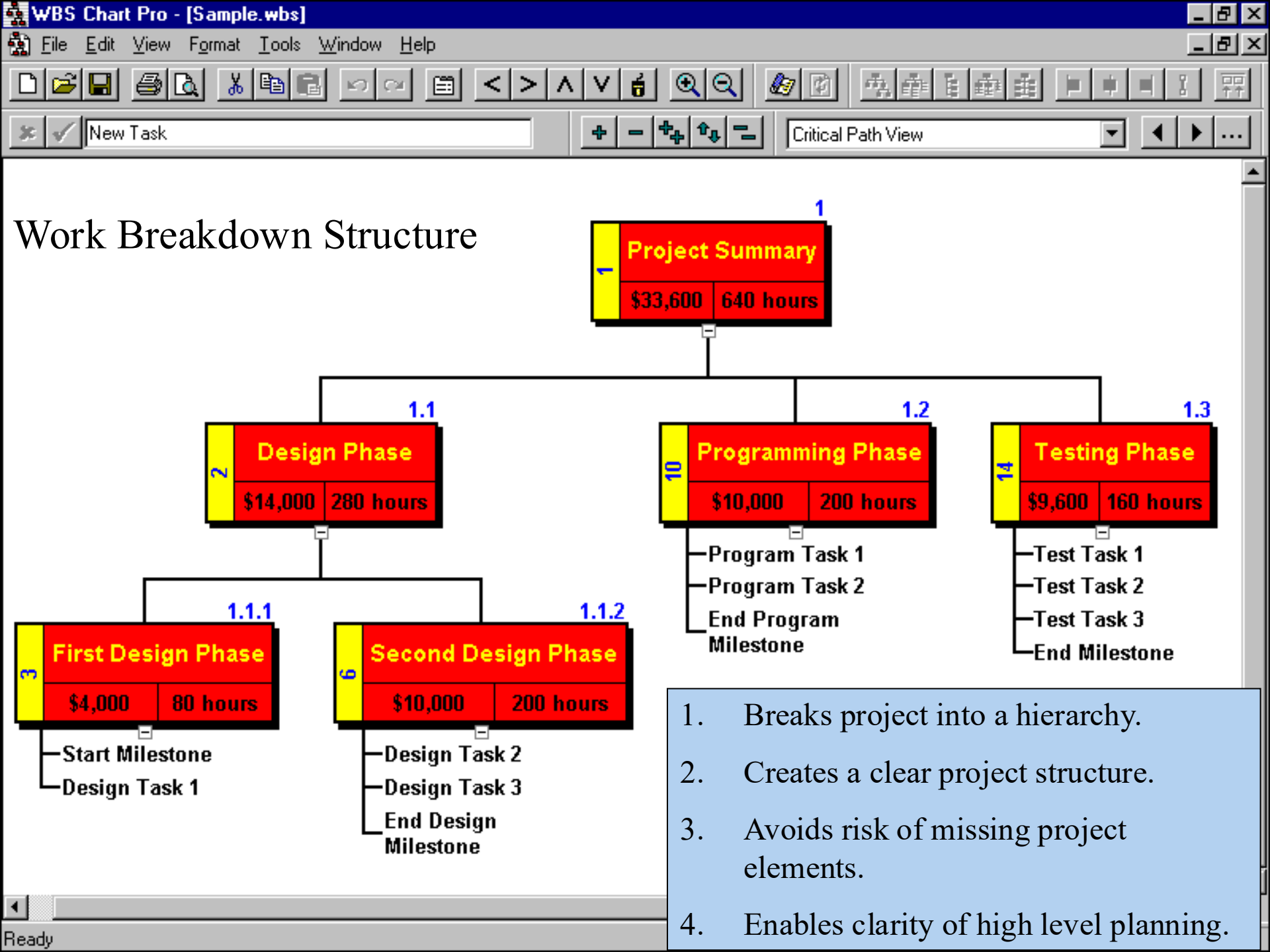
- Identify and analyze the deliverables and related work
  - The project scope statement provides the basis for the first iteration of deliverable identification
- Structure and organize the WBS according to an appropriate high-level hierarchy
  - For IT projects using an iterative system development model, a phase/iteration/discipline hierarchy most closely matches the SDLC process
- Decompose the upper WBS levels into lower-level detailed components
  - If appropriate, larger deliverables may be decomposed into smaller deliverables, all the way down to the work package level

Decomposition activities

- Develop and assign identification codes to the WBS components
  - Note that each item in the WBS has both a unique identifier (the *code or account* identifier) and a WBS code
  - The unique identifier does not change if a component is moved about in the WBS structure; however, the **WBS code** does change
- Verify that the amount of decomposition of work is necessary and sufficient
  - This can be translated into a simple concept: the 'just right' principle
  - The just right principle states that no more decomposition is needed–it would be too detailed–and any less decomposition would be too coarse
  - Insufficient decomposition leads to reduced ability to plan, manage, and control the work
  - Excessive decomposition leads to wasted effort and decreased efficiency

The Work Breakdown Structure




WBS Chart Pro - [Sample.wbs]
File Edit View Format Tools Window Help
New Task
Critical Path View
Work Breakdown Structure
1
1 Project Summary
$33,600 640 hours
1.1
2 Design Phase
$14,000 280 hours
1.2
10 Programming Phase
$10,000 200 hours
1.3
14 Testing Phase
$9,600 160 hours
Program Task 1
Program Task 2
End Program Milestone
Test Task 1
Test Task 2
Test Task 3
End Milestone
1.1.1
3 First Design Phase
$4,000 80 hours
Start Milestone
Design Task 1
1.1.2
6 Second Design Phase
$10,000 200 hours
Design Task 2
Design Task 3
End Design Milestone
1. Breaks project into a hierarchy.
2. Creates a clear project structure.
3. Avoids risk of missing project elements.
4. Enables clarity of high level planning.
Ready

Six Criteria to Test for Completeness in the WBS

- The WBS is developed as part of a Joint Planning session. But how do you know that you've done this right? Each activity must possess six characteristics to be considered complete – that is, completely decomposed. The six characteristics are
    1. Status/completion is measurable
    2. Start/end events are clearly defined
    3. Activity has a deliverable
    4. Time/cost is easily estimated
    5. Activity duration is within acceptable limits
    6. Work assignments are independent

    » Let us review each one in detail …

Six Criteria to Test for Completeness in the WBS

- **Measurable Status:** The project manager can ask for the status of an activity at any point in time during the project. If the activity has been defined properly, that question is answered easily.
  - Example: a system's documentation is estimated to be about 300 pages long and requires approximately four months of full-time work to write, here is a possible report that a developer can provide regarding the status:

    "I've written 150 pages, so I guess I am 50 percent complete."

Six Criteria to Test for Completeness in the WBS

- **Bounded with Start/end events** :
  - Each activity should have a clearly defined start & end event.
  - Once the start event has occurred, work can begin on the activity.
  - The deliverable is most likely the end event that signals work is closed on the activity.
  - **Example**: using the systems documentation example: The **start event** might be notification to the team member who will manage the creation of the systems documentation that the final acceptance tests of the systems are complete. The **end event** would be notification to the project manager that the customer has approved the systems documentation.

Six Criteria to Test for Completeness in the WBS

- **Deliverable**
  - The result of completing the work that makes up the activity is the production of a deliverable.
  - The deliverable is a visible sign that the activity is complete.
  - This could be an approving manager's signature, a physical product or document.
- **Cost/Time Estimate**
  - Each activity should have an estimated time and cost of completion.
  - Being able to do this at the lowest level of decomposition in the WBS allows you to aggregate to higher levels and the total project cost and the completion date.

Six Criteria to Test for Completeness in the WBS

- **Activity Independence**
  - It is important that each activity be independent. Once work has begun on the activity, it can continue reasonably without interruption and without the need of additional input or information until the activity is complete.
  - Though it is possible that an activity could be scheduled during different times based on resource availability.

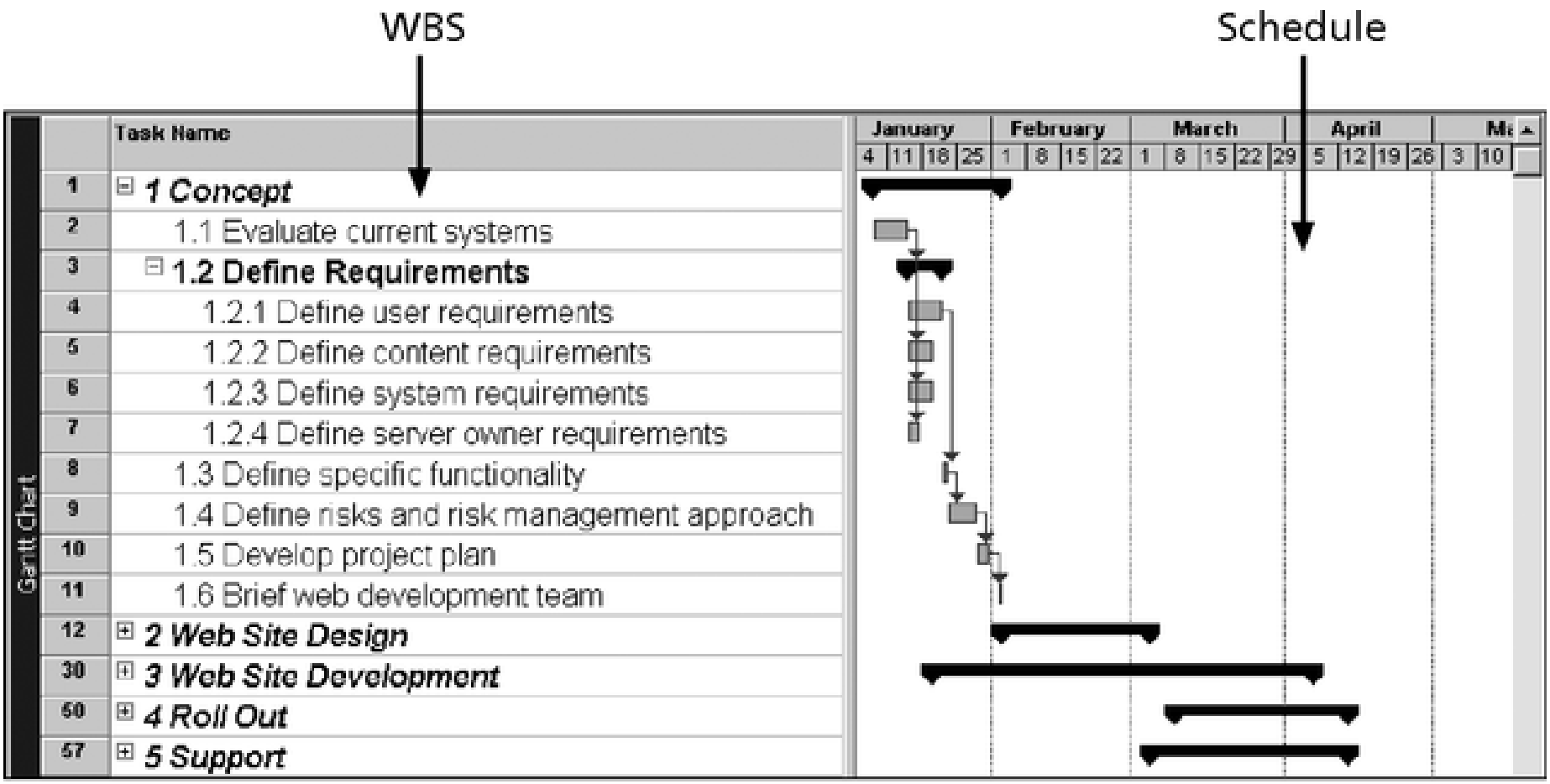
Approaches to Building the WBS

- There are many ways to build the WBS. There is no one correct way to create the WBS.
- Hypothetically, if we put each team member in a different room and asked that person to develop the project WBS, they might all come with different answers.
- There are three general approaches to building the WBS:
  1. **Noun-type** approaches.
  2. **Verb-type** approaches.
  3. **Organizational** approaches

Approaches to Building the WBS

- **Noun-type approaches**. Noun-type approaches define the deliverable of the project work in terms of the components (*physical* or *functional*) that make up the deliverable.
- **Verb-type approaches**. Verb-type approaches define the deliverable of the project work in terms of the actions that must be done to produce the deliverable. These include the *design-build-test-implement* and project *objectives* approaches.
- **Organizational approaches.** Organizational approaches define the deliverable of the project work in terms of the organizational units that will work on the project. This type of approach includes the department, process, and geographic location approaches.

WBS and Gantt Chart in Microsoft Project



The 100% rule

- The 100% rule is one of the most important principles guiding development, decomposition, and evaluation of the WBS
- Rule states that the WBS:
  - Includes 100% of the work defined by the project scope
  - Captures all internal, external, and interim deliverables in terms of work to be completed, including project management
- Rule applies at all levels within the hierarchy: the sum of the work at the "child" level must equal 100% of the work represented by the "parent"
- WBS should not include any work that falls outside the actual scope of the project, that is, it cannot include more than 100% of the work

Conventional WBS levels

- Level 1
  - Project name
- Level 2
  - Major subsystems of the project
- Level 3
  - Components/task activities of subsystems at Level 2
- Level 4
  - Subcomponents/subtasks of components/tasks at Level 3
- Level 5
  - Work packages for subcomponents/subtasks at Level 4
  - Work packages are where the actual work takes place
  - Assigned to a person and given a schedule and budget

Note that conventional WBS decomposition levels are *product-oriented*

Sample conventional WBS

Note early commitment to a specific system decomposition or architecture

Comparison of different projects requires extracting this information for each subsystem & combining

Other subsystems have same WBS pattern as sensor detection subsystem

	WBS	Task Name
1	**1**	**− Automatic Toll System**
2	**1.1**	**− System design**
3	1.1.1	Architectural design
[illegible]	**1.2**	**− Toll booth subsystem**
5	**1.2.1**	**− Sensor detection subsystem**
6	**1.2.1.1**	**− Requirements**
7	1.2.1.1.1	Requirements document
8	**1.2.1.2**	**− Analysis**
[illegible]	[illegible]	Domain model
10	1.2.1.2.2	Use-case model
11	**1.2.1.3**	**− Design**
12	1.2.1.3.1	Component design
13	**1.2.1.4**	**− Implementation**
14	1.2.1.4.1	Test case development
15	1.2.1.4.2	Coding
16	**1.2.1.5**	**− Integration**
[illegible]	1.2.1.5.1	Integration test cases
18	[illegible]	**+ Toll Authority interface**
24	**1.3**	**+ Toll Authority subsystem**

Create WBS: Outputs

- *Scope baseline*. The scope baseline is the approved version of a scope statement, work breakdown structure (WBS), and its associated WBS dictionary:
  - *Project scope statement*. Output from Define Scope process
  - *WBS*. Either graphical or tabular form
  - *WBS dictionary*. A companion document to the WBS, providing detailed information about components in the WBS, including work packages

WBS dictionary content

- **Required**
  - *Code or account identifier* – unique number assigned to a WBS component
  - *Statement of work* – scope statement for the WBS component
  - *Responsible organization* for WBS component
  - *Schedule milestones* for WBS component
- **Optional**
  - *Contract information*
  - *Quality requirements* – may be used in assessment planning
  - *Technical references* to assist in executing work
  - *Charge number*
  - *List of schedule activities*
  - *Resources required*
  - *Estimate of cost*

WBS template

Component groups with a '+' in front of them are 'rolled up' – subcomponents are hidden to reduce clutter

	WBS	Name
1	**1**	− **Project**
2	**1.1**	+ **Management**
20	**1.2**	+ **Environment**
33	**1.3**	+ **Requirements**
42	**1.4**	− **Design**
43	1.4.1	Inception phase architecture prototyping
44	**1.4.2**	− **Elaboration phase architecture baselining**
45	1.4.2.1	Architecture design modeling
46	1.4.2.2	Design demonstration planning and conduct
47	1.4.2.3	Software architecture description
48	**1.4.3**	+ **Construction phase design modeling**
51	1.4.4	Transition phase architecture design maintenance
52	**1.5**	− **Implementation**
53	1.5.1	Inception phase component prototyping
54	**1.5.2**	− **Elaboration phase component implementation**
55	1.5.2.1	Critical component coding demonstration integration (architectural baseline)
56	**1.5.3**	+ **Construction phase implementation**
61	1.5.4	Transition phase component maintenance
62	**1.6**	− **Assessment**
63	1.6.1	Inception phase assessment planning
64	**1.6.2**	− **Elaboration phase assessment**
65	1.6.2.1	Test modeling
66	1.6.2.2	Architecture test scenario implementation
67	1.6.2.3	Demonstration assessment and release descriptions
68	**1.6.3**	+ **Construction phase assessment**
72	**1.6.4**	+ **Transition phase assessment**
74	**1.7**	+ **Deployment**

Activity or task definition

	WBS	Name
1	**1**	**– Automatic Toll Booth System**
42	**1.4**	**– Design**
43	1.4.1	Inception phase architecture prototyping
44	**1.4.2**	**– Elaboration phase architecture baselining**
45	**1.4.2.1**	**– System architecture definition**
[illegible]	[illegible]	Define high-level hardware architecture
47	1.4.2.1.2	Define high-level communications architecture
48	**1.4.2.2**	**– Architecture design modeling**
49	1.4.2.2.1	Perform ABD (architecture-based design)
50	[illegible]	Perform quality attribute analysis (ATAM)
[illegible]	1.4.2.2.3	Perform cost/benefit analysis (CBAM)
[illegible]	1.4.2.2.4	Document software architecture
53	**1.4.2.3**	**– Design demonstration planning and conduct**
54	1.4.2.3.1	Select architectural baseline critical use-cases
55	1.4.2.3.2	Select architectural baseline critical functionality
56	[illegible].2.3.3	Design architectural baseline components
[illegible]	**1.4.3**	**+ Construction phase design modeling**
60	1.4.4	Transition phase architecture design maintenance
61	**1.5**	**– Implementation**
62	1.5.1	Inception phase component prototyping
63	**1.5.2**	**– Elaboration phase component implementation**
64	**1.5.2.1**	**– Critical component coding demonstration integration (architectural baseline)**
65	1.5.2.1.1	Implement architectural baseline components
66	1.5.2.1.2	Integrate architectural baseline components
[illegible]	**1.5.3**	**+ Construction phase implementation**
[illegible]	1.5.4	Transition phase component maintenance
73	**1.6**	**– Assessment**
74	1.6.1	Inception phase assessment planning
75	**1.6.2**	**– Elaboration phase assessment**
76	1.6.2.1	Select and define architectural baseline tests
77	1.6.2.2	Implement architecture test scenario
78	1.6.2.3	Test architectural baseline

Added *System architecture definition* WBS component

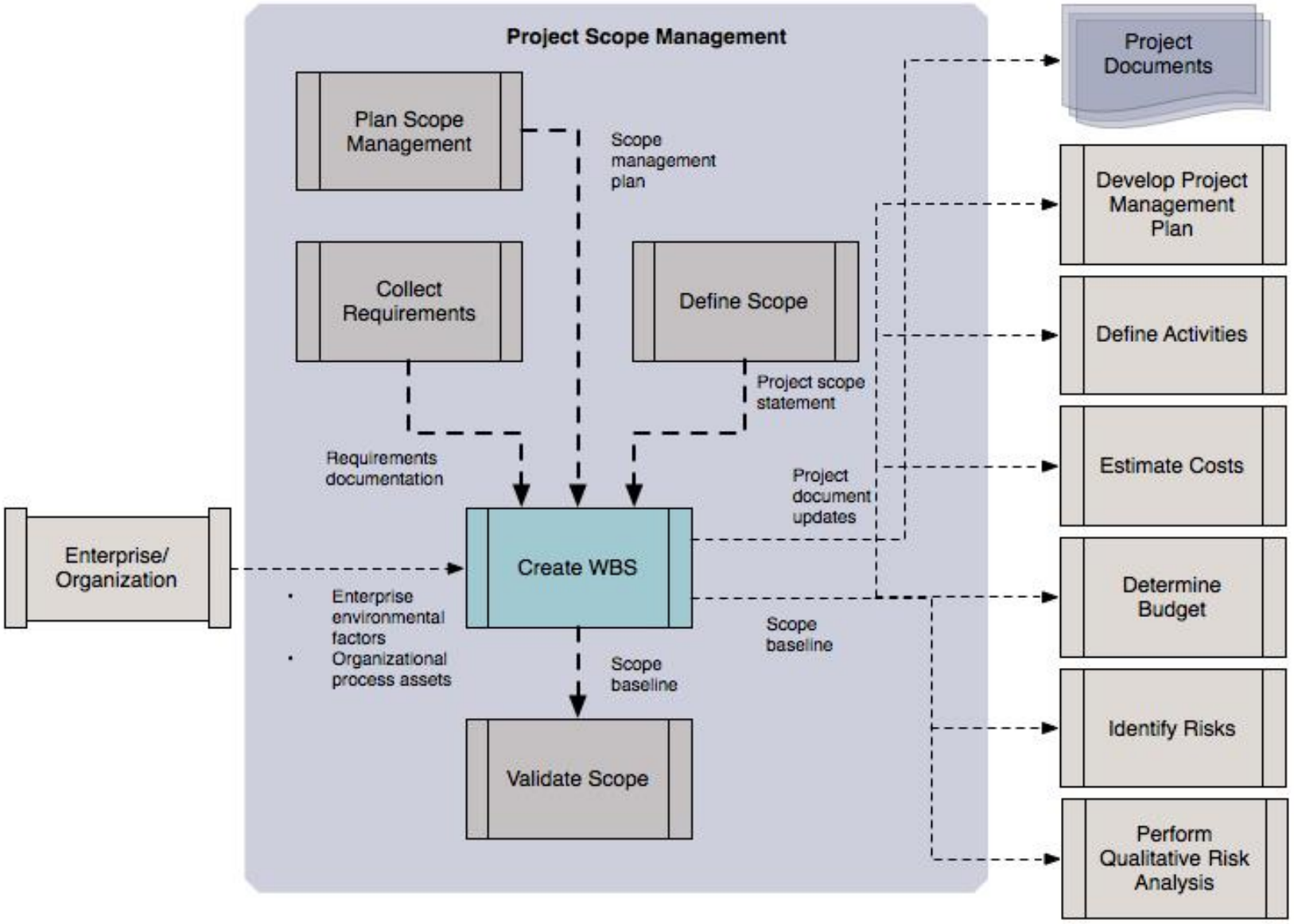
Note expansion and detailing of WBS template *Architecture design modeling* entry; renamed *Software architecture description* to *Document software architecture*

Note expansion and detailing of WBS template *Design demonstration planning and conduct* entry

Note expansion and detailing of WBS template *Critical component coding demo integration* entry

Note rework of WBS template *Elaboration phase assessment* entry

Create WBS: Data Flow Diagram



Agile Perspective: Create WBS

Valuable concepts and tools

- The *Create WBS* process is among the least compatible with a complex (or agile) project perspective
  - ➢ It encourages solidifying the scope of the project in a complex, difficult to change artifact, the WBS
  - ➢ Once an artifact is created, it assumes an authority that may not be justified
  - ➢ Most WBSs are out-of-date shortly after being created
  - ☛ People are reluctant to toss something that has taken so much effort
- Nevertheless, the process of thoughtful decomposition of the product into smaller, more manageable pieces is certainly compatible with an adaptive/agile methodology
- Adaptive/agile limits high-level decomposition to the product roadmap, and defers low-level decomposition to the release and iteration (sprint) level

WBS – Basis of Many Things

- Network scheduling
- Costing
- Risk analysis
- Organizational structure
- Control
- Measurement

The MS-Project Process

- Move WBS into a Project outline (in Task Sheet)
- Add resources (team members or roles)
- Add costs for resources
- Assign resources to tasks
- Establish dependencies
- Refine and optimize
- Create baseline
- Track progress (enter actuals, etc.)

Defining Task Sets

- Determine type of project
- Assess the degree of rigor required
- Identify adaptation criteria
- Select appropriate software engineering tasks
- Task Set Refinement
  - Use Work Breakdown Structure to determine tasks
- Define a Task Network
  - Use a Gantt Chart and/or PERT to document